

API

+MICROSERVICES

Le guide essentiel pour créer une
plateforme d'API professionnelle

TABLE DES MATIÈRES

3	Introduction
4	Présentation des API web
11	Les avantages de l'évolution vers une activité centrée sur les API
14	Commencer à réfléchir en termes de plateforme d'API
24	L'émergence des microservices
30	Les composants d'une plateforme d'API
36	Planifier l'évolution vers une activité centrée sur les API
47	Conclusion

INTRO- DUCTION

Les API se trouvent au point de convergence des activités, des produits et des technologies. Elles améliorent l'expérience client, renforcent les relations commerciales et stimulent l'innovation interne.

Un responsable technique doit faire de nombreux choix en matière de langages de programmation, de bibliothèques et d'infrastructures. Mais, contrairement à tous ces éléments, les API ont un impact direct sur la rapidité de la production de logiciels au sein d'une entreprise. Par conséquent, un responsable ne doit pas laisser aux développeurs le soin d'établir la stratégie de son entreprise en matière d'API. Il faut plutôt adopter une approche mûrement réfléchie et bien planifiée à l'échelle de l'entreprise.

Cet Ebook s'adresse aux responsables techniques chargés d'établir un nouveau programme d'API pour leur entreprise ou de consolider un programme existant. Il propose des informations exploitables et des facteurs déterminants basés sur des pratiques établies au sein d'entreprises suivant des programmes d'API qui ont fait leurs preuves.

CHAPITRE

1

PRÉSENTATION DES API WEB

Qu'est-ce qu'une API web ?

Pour rappel, une interface de programmation d'applications (API) spécifie les types d'interactions entre systèmes et composants logiciels. Une API web étend cette interaction au-delà d'une simple application en utilisant le protocole réseau HTTP, le langage d'Internet.

Une API web ne doit pas forcément être de type RESTful. Elle ne doit pas obligatoirement prendre en charge le protocole SOAP. Elle ne doit pas nécessairement utiliser JSON, XML ou OAuth, ou être conçue dans un langage de programmation ou une infrastructure spécifique. Inutile de lui associer des URL attrayantes. Les API web peuvent présenter toutes ces caractéristiques, certaines seulement ou aucune. Voici le seul impératif : une API web doit permettre à un programme ou un composant logiciel d'interagir avec un homologue de manière reproductible sur HTTP.

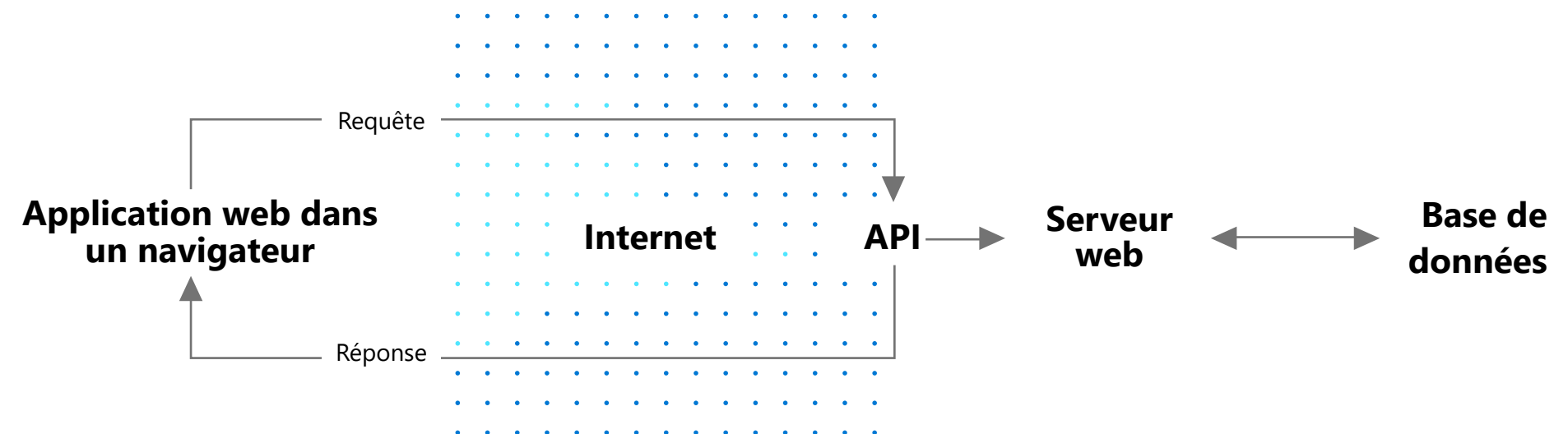


Schéma illustrant une application web simple qui renvoie les données extraites d'une base de données via une API.

L'ESSOR DES API WEB ET MOBILES

Voici les trois raisons principales pour lesquelles les API web ont gagné en popularité au cours des dix dernières années :

Simplicité

Les API web communiquent via un protocole simple, à savoir HTTP, le langage d'Internet. Cela simplifie l'intégration à partir de n'importe quel langage de programmation. Aucune bibliothèque spécifique n'est requise.

Réduction des coûts

Comparé à la génération précédente de technologies, qui nécessitaient des intergiciels (middleware) coûteux et des formations, les technologies open source permettent de réduire les coûts de conception d'API web.

Hausse de la demande

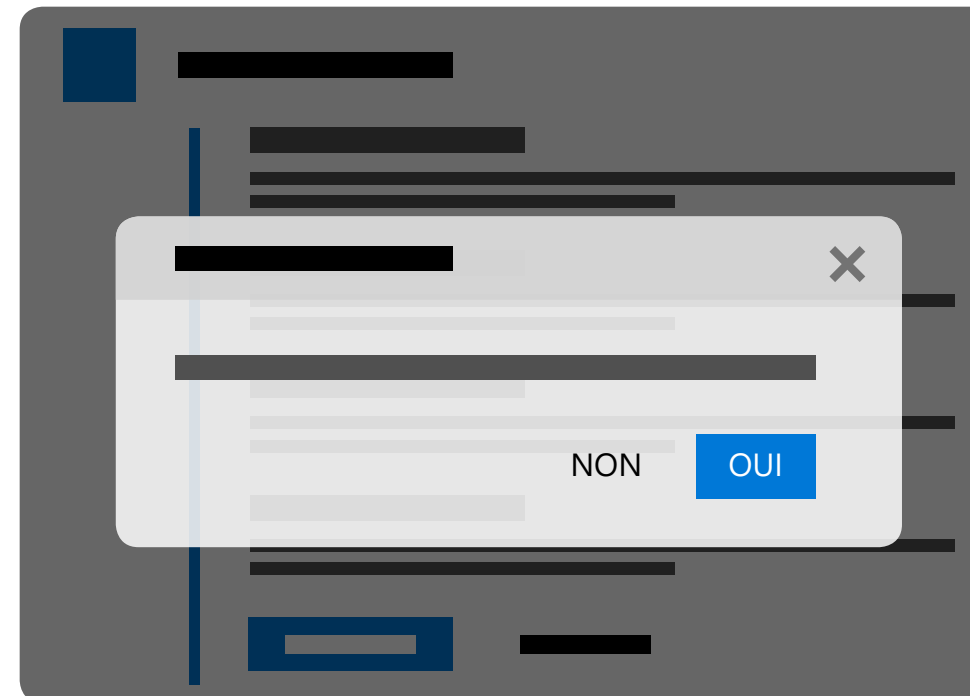
En raison des impératifs de mobilité et d'accès à tout moment et en tout lieu, on est passé d'opérations en langage HTML à la séparation des données et des fonctionnalités au niveau de la couche présentation. Cela concerne autant les applications monopages (SPA) que les applications mobiles natives et web.

Les API en tant que fonctionnalités

LES PRODUITS LES PLUS IMPORTANTS

de votre entreprise, ce sont les capacités professionnelles et techniques que proposent vos applications. Associées dans des solutions, ces fonctionnalités permettent à vos clients, vos partenaires et votre personnel de résoudre leurs problèmes quotidiens et d'atteindre les résultats escomptés.

Avec l'émergence des plateformes de messagerie, des bots et des interfaces vocales, les choses sont en train de changer radicalement. Nous commençons à nous rendre compte qu'il faut revoir les priorités et proposer des fonctionnalités via des API, qui seront ensuite intégrées à ces plateformes. Plutôt que d'accéder à une application pour réaliser une tâche, l'utilisateur se voit désormais proposer directement des applications via des plateformes tierces.



Les applications sont désormais mises à la disposition de l'utilisateur via des plateformes de messagerie comme Slack, Cisco Teams et Microsoft Teams.

Les API web proposent des fonctionnalités jouant des rôles très divers

API PRIVÉES

Dissimulées derrière le pare-feu des entreprises, les API privées permettent d'exécuter les opérations quotidiennes. Cela va des API qui font fonctionner les applications mobiles aux API utilisées en interne par le personnel. Voici quelques champs d'application : gestion des stocks, réservation de chambres d'hôtel et gestion des comptes.

API PARTENAIRES

Les API partenaires, comme celles proposées par Sabre et Capital One, sont orientées vers l'ensemble des partenaires de développement d'une entreprise. En limitant l'accès d'une API à ses partenaires, une entreprise peut contrôler étroitement son utilisation (par qui et comment elle est utilisée en externe).

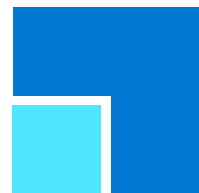
API SAAS

Des sociétés comme GitHub, Stripe et SendGrid tirent parti des API SaaS pour accéder à leurs données hébergées. Elles peuvent choisir d'utiliser le modèle SaaS via l'interface utilisateur existante, via l'API ou bien en associant les deux. Les fournisseurs SaaS utilisent des API pour fidéliser les clients en les aidant à intégrer les logiciels SaaS à leur workflow quotidien.

API DE PLATEFORME

Les API de plateforme étendent la portée des fournisseurs SaaS comme Microsoft Dynamics, Fitbit et API2Cart en réunissant plusieurs acteurs d'un marché. En allant au-delà de la relation type fournisseur-client propre au modèle SaaS, les plateformes offrent souvent une valeur ajoutée grâce à l'innovation et à un réseau global plus vaste.

Les API en tant que produits



Non seulement les API web servent à proposer des fonctionnalités au sein d'une entreprise, mais elles deviennent également des produits à proprement parler. Voici un exemple récent illustrant cette tendance : Twilio a commencé par offrir un moyen simple d'automatiser les interactions vocales et textuelles. Au fil du temps, elle s'est développée pour prendre en charge toute une gamme de capacités, allant du fax à la messagerie électronique. En appliquant des techniques rigoureuses de gestion des produits, Twilio a adopté ce qui s'apparente à un produit orienté développeurs. Elle figure parmi la poignée de sociétés à avoir fait un premier appel public à l'épargne sur le marché boursier américain alors que son offre se limitait à un produit basé sur les API.

Les entreprises peuvent suivre les mêmes techniques qui ont conduit Twilio, Stripe et d'autres sociétés à commercialiser des produits reposant sur les API. Les entreprises désireuses de résoudre les problèmes qui intéressent le plus les développeurs choisissent de concevoir des API visant à atteindre les résultats escomptés par les utilisateurs. Elles cherchent à appliquer la propriété des produits à leurs API, à solliciter l'avis des parties prenantes et à améliorer en permanence la conception et la documentation des API pour en faciliter l'adoption.

Les entreprises qui ne traitent pas leurs API en tant que produits concevront des API qui se limitent à sérialiser des données sur HTTP ou à assurer l'intégration d'un système à l'autre. Ce style d'API a sa place dans certains contextes, mais les entreprises ne bénéficieront pas des économies d'échelle qu'offre le concept d'API en tant que produit, à savoir la réutilisabilité, la vitesse de réalisation et l'homogénéité de l'expérience client.

Les business models reposant sur les API

Une stratégie efficace doit tout d'abord définir les objectifs du programme d'API.

Objectifs courants :

Accélération

de la stratégie mobile en rendant les données et les services plus accessibles

Adaptation

à l'évolution de la relation client, qui va au-delà des interactions web et mobiles, et s'étend à de nouveaux appareils et expériences (omnicanal)

Transformation

de l'intégration avec des partenaires en améliorant l'efficacité et en libérant des ressources

Stimulation

de l'innovation technique et commerciale en réduisant les obstacles à la commercialisation de nouvelles solutions

Élimination

des silos pour faciliter le partage de données et le rendre plus cohérent entre les systèmes et les équipes internes

Augmentation

directe ou indirecte des revenus en fidélisant les clients grâce à une intégration plus étroite

Voici trois exemples
illustrant comment des entreprises
utilisent les API pour atteindre un ou
plusieurs de ces objectifs :

FEDEX

L'API FedEx améliore l'efficacité des partenaires tout en simplifiant leur propre système logistique en s'appuyant sur celui de FedEx. Le lancement initial de cette API a permis à des entreprises de gagner en visibilité sur la logistique d'expédition, un processus opaque il y a encore quelques années. Maintenant, chaque site FedEx s'appuie efficacement sur une API qui permet à quiconque de télécharger partout des documents pour les imprimer à proximité.

BEST BUY

Best Buy a mis à disposition du public une API pour s'adapter à l'évolution de la relation client. Cette entreprise possède des magasins gigantesques abritant de vastes stocks, qui pâttissent de la tendance du « showrooming ». Concrètement, les clients potentiels entrent dans un magasin, examinent la marchandise et dès qu'ils trouvent l'article dont ils ont besoin, ils visitent un site comme Amazon pour rechercher une meilleure offre et passer commande. Best Buy a décidé de rendre son API accessible aux développeurs, ce qui a eu pour effet de mettre d'énormes quantités de données à disposition de tiers. La société a été en mesure de constituer une communauté active ayant à cœur d'explorer les données et leurs différentes utilisations.

NETFLIX

Lorsque la société Netflix a commencé à envoyer des DVD par la poste en 1997, elle était en concurrence avec Blockbuster et les opérateurs de réseau câblé. Au cours des années suivantes, Netflix a peu à peu phagocyté le marché de la location de DVD tout en se préparant à lancer son service de streaming en février 2007. Aujourd'hui, difficile de trouver un appareil multimédia qui ne prend pas en charge la diffusion en continu Netflix. Ce résultat est dû en grande partie à une initiative stratégique en matière d'API. Netflix veille à ce que son API s'intègre aisément à tous les appareils qui peuvent être reliés à un écran. La prochaine fois que vous utiliserez Netflix, pensez à chacune des interactions engagées jusqu'au moment de la diffusion d'un spectacle ou d'un film. Chaque interaction est optimisée par une fonctionnalité offerte par son API d'intégration d'appareils.

CHAPITRE

2

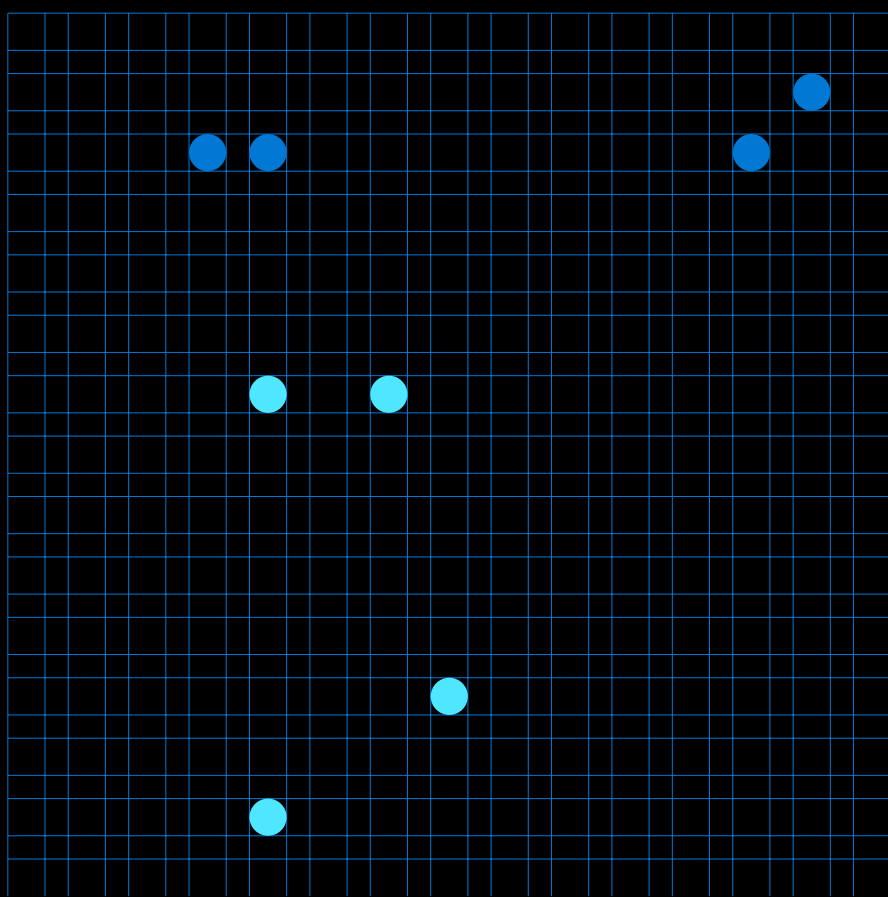
LES AVANTAGES DE L'ÉVOLUTION VERS UNE ACTIVITÉ CENTRÉE SUR LES API

Les API encouragent la réutilisation et accélèrent les processus

Les API constituent un catalyseur de choix pour les entreprises car elles peuvent être réutilisées à l'infini. Par ailleurs, elles accélèrent les opérations en évitant aux développeurs de réécrire le même code à maintes reprises. Ce qu'une seule équipe a créé peut servir à de nombreux développeurs pour produire des solutions utilisées par des centaines, voire des milliers de personnes.

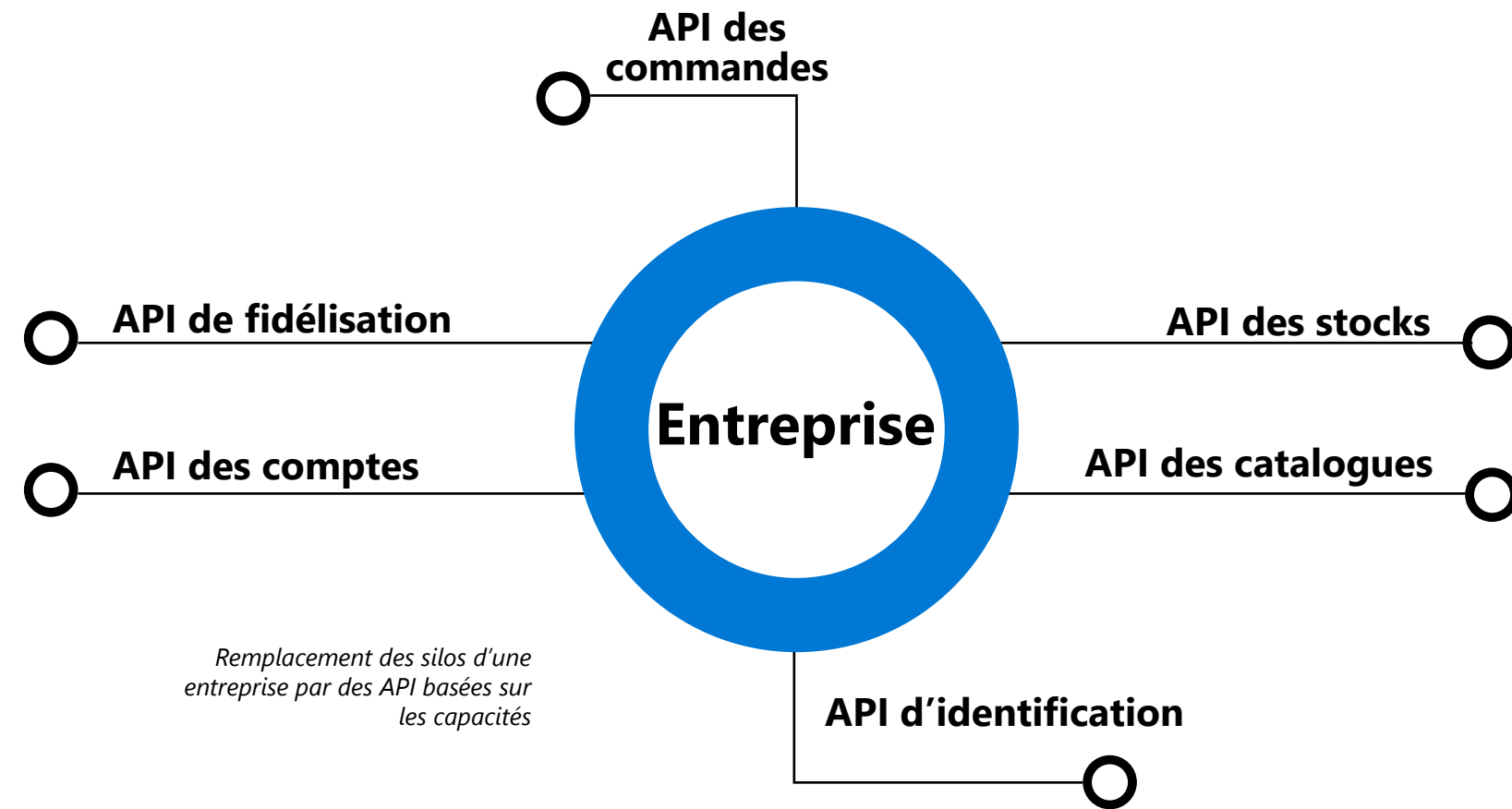
La réutilisation des API dépend souvent des développeurs internes. Au lieu de partager des bibliothèques dans un langage de programmation spécifique, les API permettent de réutiliser du code dans un large éventail de langages. Certaines entreprises sont allées encore plus loin en calculant le nombre de lignes qu'il n'était plus nécessaire d'écrire après l'adoption d'une API.

Les API produites en interne ne serviront pas toutes à des centaines de clients, mais il suffit qu'une de vos équipes réinstalle une fonctionnalité (avec un comportement légèrement différent) pour observer un impact négatif sur vos clients et réduire la réactivité de votre entreprise.



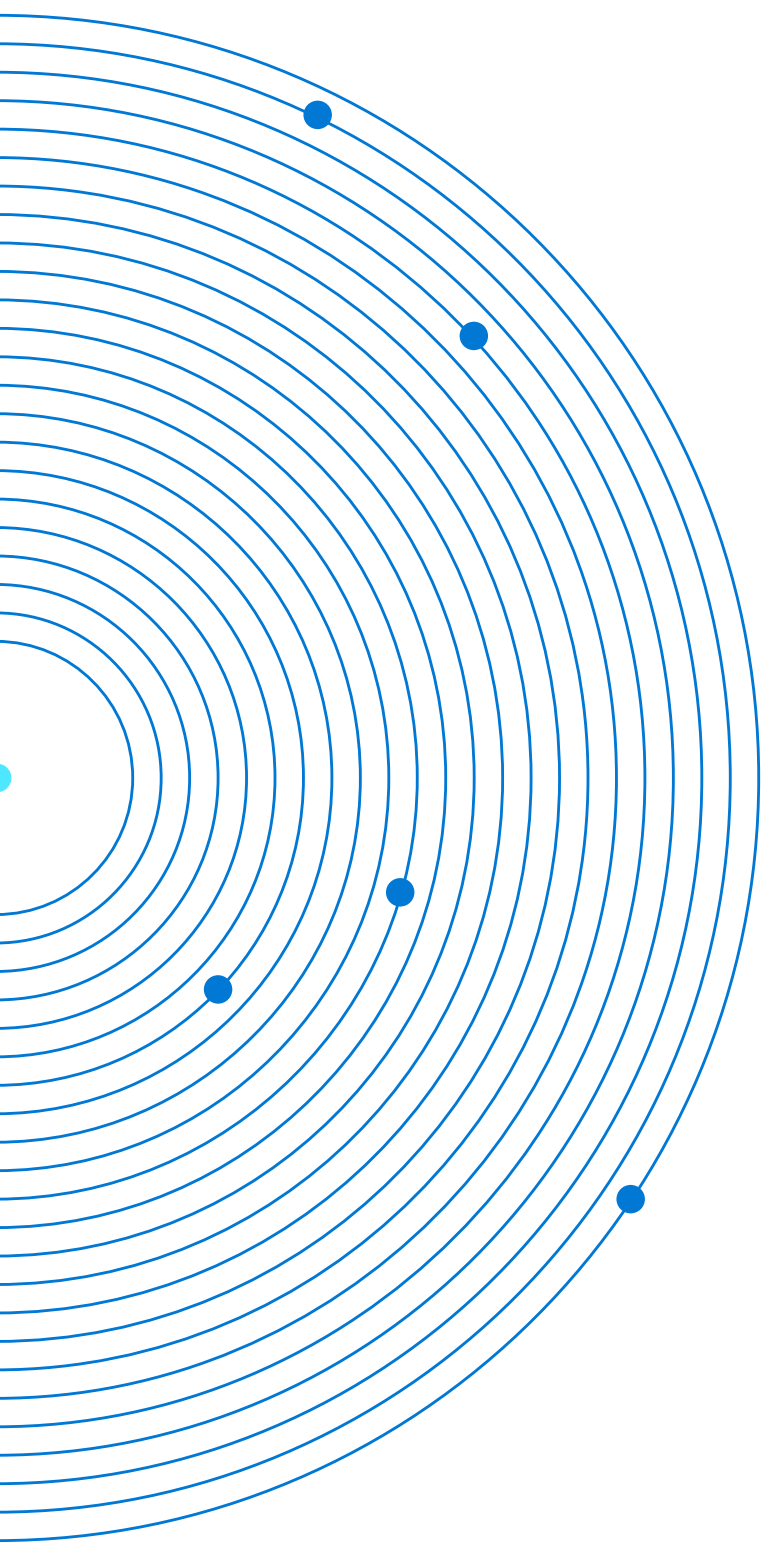
Les API éliminent les silos

Les entreprises rencontrent toujours des difficultés avec les applications cloisonnées qui stockent des données et renferment des fonctionnalités. Dans le cadre d'une activité centrée sur les API, ce ne sont plus les applications cloisonnées qui hébergent les données et les capacités, mais les API. Ainsi, les entreprises deviennent plus modulaires. Elles sont en mesure de dissocier et de regrouper leurs fonctionnalités pour innover et interagir avec les clients de façon inédite et attrayante.



Un programme d'API efficace permet d'éliminer ces systèmes cloisonnés en les divisant en problèmes à solutionner (parfois appelés « contextes délimités »).

Ces domaines restreints offrent une API claire qui fournit les capacités et les données indispensables pour surmonter une difficulté commerciale spécifique. Des solutions sont ensuite conçues sur ces API pour répondre aux besoins des activités internes, des partenaires, des développeurs d'applications et des fournisseurs tiers.



Les API assurent la cohérence de l'expérience client

Vous avez sans doute observé des expériences client inégales. Parfois, une application mobile ne propose pas les mêmes fonctions que le site web. Autre possibilité : le site web ne fonctionne peut-être pas exactement comme une application mobile mise à jour récemment. Quelle que soit la cause, l'expérience n'est pas homogène, ce qui peut entraîner une perte de clients.

Il faut rappeler qu'une API est là pour aider les clients à réaliser une tâche spécifique. Soit il faut utiliser l'API pour démarrer et terminer une procédure (ou exécuter une partie de la tâche), soit il s'agit de relier des systèmes et des entreprises via une communication entre machines. Par conséquent, les API doivent être conçues pour atteindre des objectifs sans jamais négliger l'expérience client. N'oubliez pas qu'une API mal conçue peut avoir un impact négatif sur un grand nombre de développeurs et un nombre encore plus important de clients.

Les API constituent l'opportunité ultime en informatique

Quels que soient le business model et la taille de votre entreprise, les API deviennent le nec plus ultra pour le service informatique. Les entreprises sont en mesure de concevoir des API reposant sur des fonctionnalités et des tâches spécifiques, puis d'adapter leur installation pour intégrer les systèmes appropriés et les sources de données nécessaires. Même si on en est encore au stade de l'esquisse, ces API représentent l'avenir de l'informatique en entreprise.

Grâce à des API bien conçues qui représentent l'état souhaité, des systèmes qu'il était auparavant impossible de mettre hors service sont désormais dissimulés derrière ces API. Les entreprises qui adoptent une approche axée sur les API ont plus de chances de migrer vers des systèmes récents et d'ajouter de nouvelles fonctionnalités, tout en mettant hors service les systèmes anciens actuellement utilisés au quotidien. Définissez le cap à suivre dans le cadre de vos initiatives en matière d'API. Ensuite, déterminez les processus et les efforts que doit consentir votre entreprise pour toucher au but.

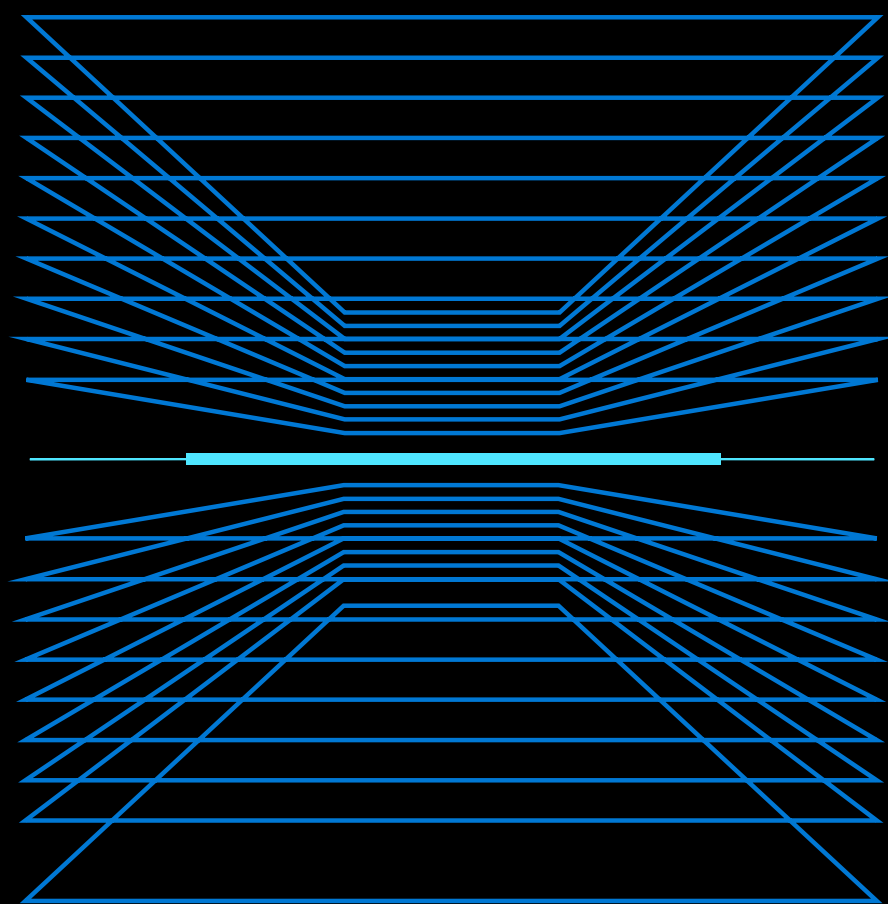
CHAPITRE

3

COMMENCER À RÉFLÉCHIR EN TERMES DE PLATEFORME D'API

Qu'est-ce qu'une plateforme d'API ?

Une plateforme conçoit un écosystème destiné aux entreprises, aux clients et aux partenaires. Elle connecte toutes les activités de l'entreprise, en interne comme en externe. Jusqu'à récemment, la plupart des plateformes étaient conçues par des fournisseurs qui connectaient un marché à plusieurs facettes, comme la gestion de la chaîne d'approvisionnement, la gestion de la relation client (CRM) et la gestion intégrée (ERP). Désormais, les plateformes d'API émergent pour développer des solutions internes, partenaires et publiques en vue de répondre à la demande du marché.



Les quatre composants d'une plateforme d'API

Les API fournissent des interfaces d'accès aux données et aux comportements pour proposer des fonctionnalités numériques, généralement sur HTTP. Votre portefeuille d'API s'étend sans doute sur plusieurs secteurs d'activité pour répondre à toute une série de besoins (internes, clients et partenaires).

LES PLATEFORMES D'API PERFORMANTES VONT AU-DELÀ DES LIMITES DES API SIMPLES

Une plateforme d'API efficace englobe les éléments suivants :

1.

Combinaison d'API privées, partenaires et publiques

2.

Plusieurs styles d'API

3.

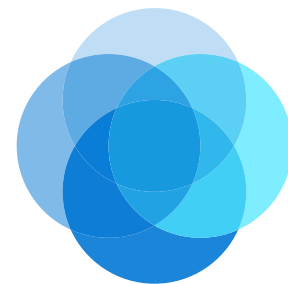
Notification d'événements pour réagir au moindre changement

4.

Diffusion de messages pour constituer le réseau dorsal des données

Étudions chacun de ces éléments et déterminons comment ils transforment les fonctionnalités commerciales et techniques en plateforme d'API robuste.

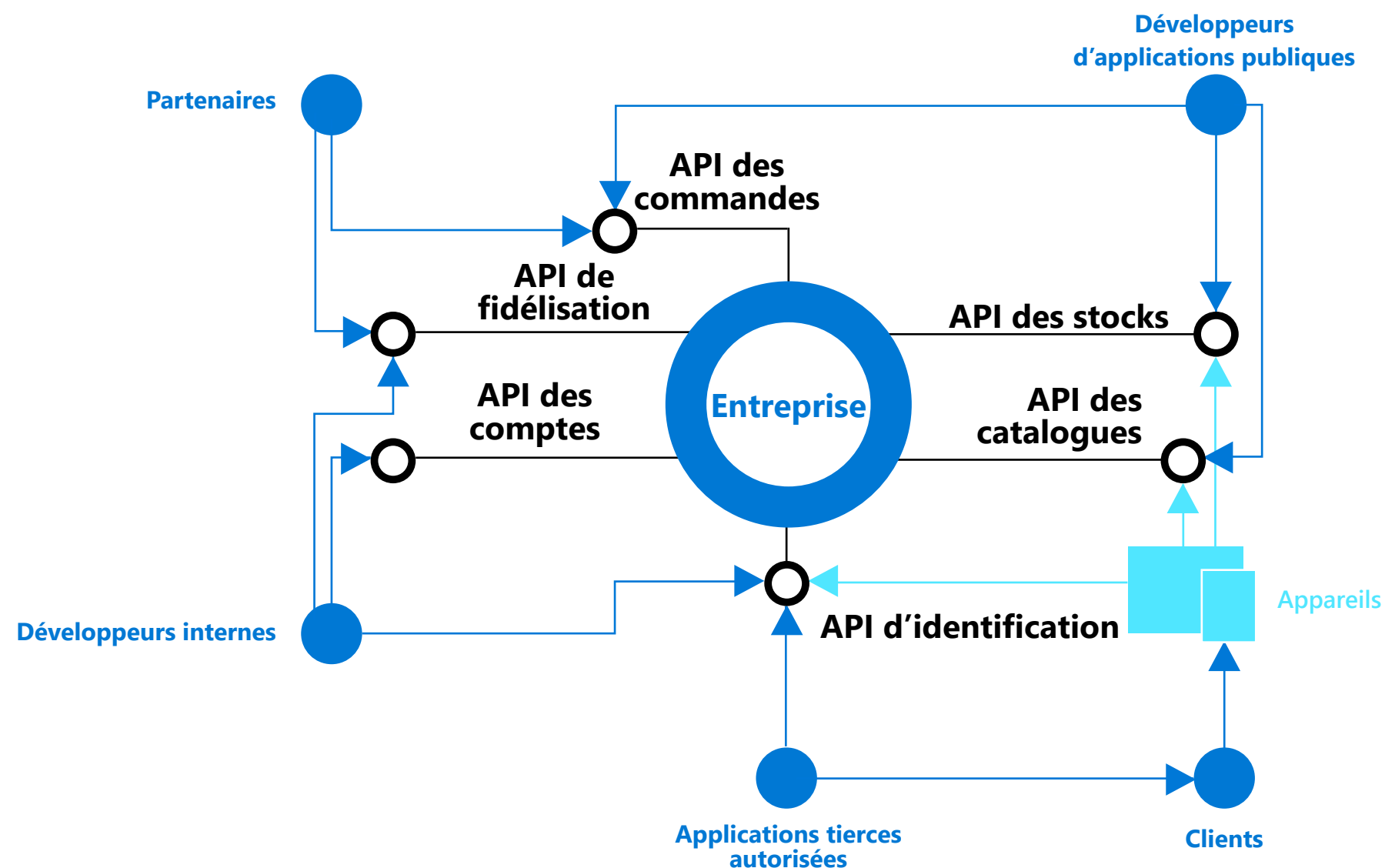
Une plateforme englobe des API privées, partenaires et publiques



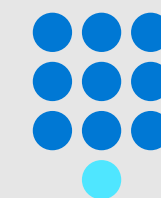
Les API publiques développées par des sociétés comme Twilio et Stripe font souvent la une des journaux, mais la plupart des entreprises limitent l'exposition de leurs API publiques à un ensemble limité d'offres. Des établissements bancaires comme Capital One proposent quelques API en tant que produits aux développeurs. Ces API sont confinées à un environnement de test, et l'accès à l'environnement de production est réservé aux acteurs qui souhaitent s'engager dans un partenariat formel. D'autres sociétés comme Best Buy facilitent l'utilisation de leurs API.

Les API ne doivent pas toutes être externalisées pour être accessibles au public ou aux partenaires. Les API privées permettent à une entreprise de décomposer ses opérations en capacités numérisées via des API basées sur des systèmes internes. Certaines plateformes d'API actuelles sont constituées exclusivement d'API privées accessibles uniquement au sein de l'entreprise.

On estime parfois qu'il y a dix fois plus d'API privées que d'API accessibles au public. Ce chiffre est probablement bien plus élevé. En effet, la plupart des entreprises préfèrent débiter avec des API privées avant de les rendre accessibles aux partenaires et aux développeurs publics.



Un portefeuille d'API répond aux besoins de l'entreprise, des partenaires, des clients et des développeurs d'applications publiques.



Les API privées représentent la majeure partie du portefeuille d'API, mais les entreprises ont souvent besoin d'assurer une intégration avec leurs partenaires. Les opérations auparavant réalisées à l'aide de fichiers de commandes et d'intégrations SOAP sont désormais effectuées via des API web. Ces API web orientées partenaires forment un sous-ensemble du portefeuille d'API, proposées en tant que produits et prises en charge pour s'intégrer efficacement et faciliter leur adoption par les partenaires.

À ce jour, la plupart des API se concentrent sur l'intégration d'un ou de plusieurs systèmes. Ces API font office de ciment qui oriente les opérations quotidiennes et épaulle le personnel. Toutefois, les API sont bien plus qu'une technologie d'intégration. Elles numérisent des capacités qui représentent l'activité. Ces fonctionnalités numériques peuvent être utilisées en interne, pour stimuler l'intégration avec des partenaires, mais aussi pour faire fonctionner des applications web et mobiles.

Une plateforme doit englober plusieurs styles d'API

Aujourd'hui, les trois styles d'API les plus courants sont REST, GraphQL et gRPC. Chacun a ses points forts et ses points faibles, résumés ici :

REST

Acronyme de Representational State Transfer. En l'an 2000, le Dr Roy Fielding a détaillé ce style d'architecture dans sa thèse de doctorat, qui décrit les décisions et les contraintes liées à la spécification HTTP. Même s'il ne s'agit pas d'une norme, ce style sert de référence pour concevoir les API web.

Au fil du temps, REST est devenu synonyme de JSON sur HTTP pour certains, généralement via le modèle CRUD (créer, lire, mettre à jour et supprimer). Même si c'est un modèle courant, il est possible de suivre différentes approches pour créer des API web sur HTTP. La gestion des API REST est aisée car chaque fonctionnalité possède sa propre URL, qui peut être sécurisée et surveillée via la gestion des API (APIM), comme nous le verrons plus loin. Par conséquent, le fait de proposer une API REST est un excellent point de départ, dans la mesure où cela permet à une application ou un script d'automatisation d'interagir avec votre API sur HTTP.

GraphQL

Cette méthode a été développée par Facebook pour optimiser les interactions basées sur HTTP entre l'application mobile et le back end graphique. Depuis lors, c'est l'option préférée des développeurs d'applications frontales qui cherchent une façon cohérente d'exprimer des requêtes pour extraire exactement les données souhaitées. Ce style est également utilisé pour agréger plusieurs API REST dorsales dans une seule et même requête. Une demande GraphQL ressemble à un appel de procédure distante (RPC) SOAP, avec un point de terminaison unique qui accepte un format de requête spécifique. Il est donc plus difficile de sécuriser, surveiller et gérer une requête de ce type car elle est traitée comme une seule URL par la couche APIM.

gRPC

Ce style a été développé chez Google en tant que méthode hautes performances pour envoyer des requêtes RPC entre différents services. gRPC repose sur la spécification HTTP/2, qui prend en charge les améliorations apportées en termes de performances et les communications de multiplexage. Ce protocole est optimisé pour les communications interservices au cœur d'une architecture basée sur les microservices.

En fonction des styles d'API choisis, votre entreprise prendra en charge différents cas d'utilisation :

SOLUTIONS

STYLES ET PROTOCOLES D'API

Application web d'achat	API REST d'achat	API GraphQL d'achat
API de marché tierce	API REST de commande	API gRPC de gestion des stocks
Compétence d'achat Alexa (voix)	API gRPC de gestion des stocks	API gRPC de gestion des catalogues

Les entreprises peuvent envisager plusieurs options pour exposer leurs capacités basées sur les API. Selon leurs choix, les plateformes d'API répondront aux besoins spécifiques des développeurs en proposant un ou plusieurs styles d'API.

Dans la plupart des entreprises, il est judicieux de choisir REST en tant que style d'API de base. En effet, il simplifie la gestion, renforce la sécurité et propose un historique des outils. Le cas échéant, gRPC peut être utilisé pour optimiser la communication entre divers services internes. GraphQL est le complément idéal de REST pour les API frontales et de génération de rapports qui requièrent des optimisations afin d'améliorer l'expérience client.

Les notifications d'événements de plateforme favorisent l'extensibilité



Voici l'un des éléments décisifs d'une plateforme d'API : la prise en charge des notifications d'événements. Chaque fois qu'une fonctionnalité d'API crée ou modifie des données, cette opération est liée à moins un événement. Cela intéresse un autre système ou une autre solution de recevoir une notification au moment où cet événement se produit.

Les événements permettent de faire reposer des solutions sur la plateforme sans même qu'elle s'en aperçoive. Ces solutions émergentes reçoivent une notification en cas de modifications de l'état. Par ailleurs, les systèmes n'ont plus besoin d'interroger en permanence une API pour déterminer si des données ont changé, ce qui permet d'optimiser l'utilisation des ressources.

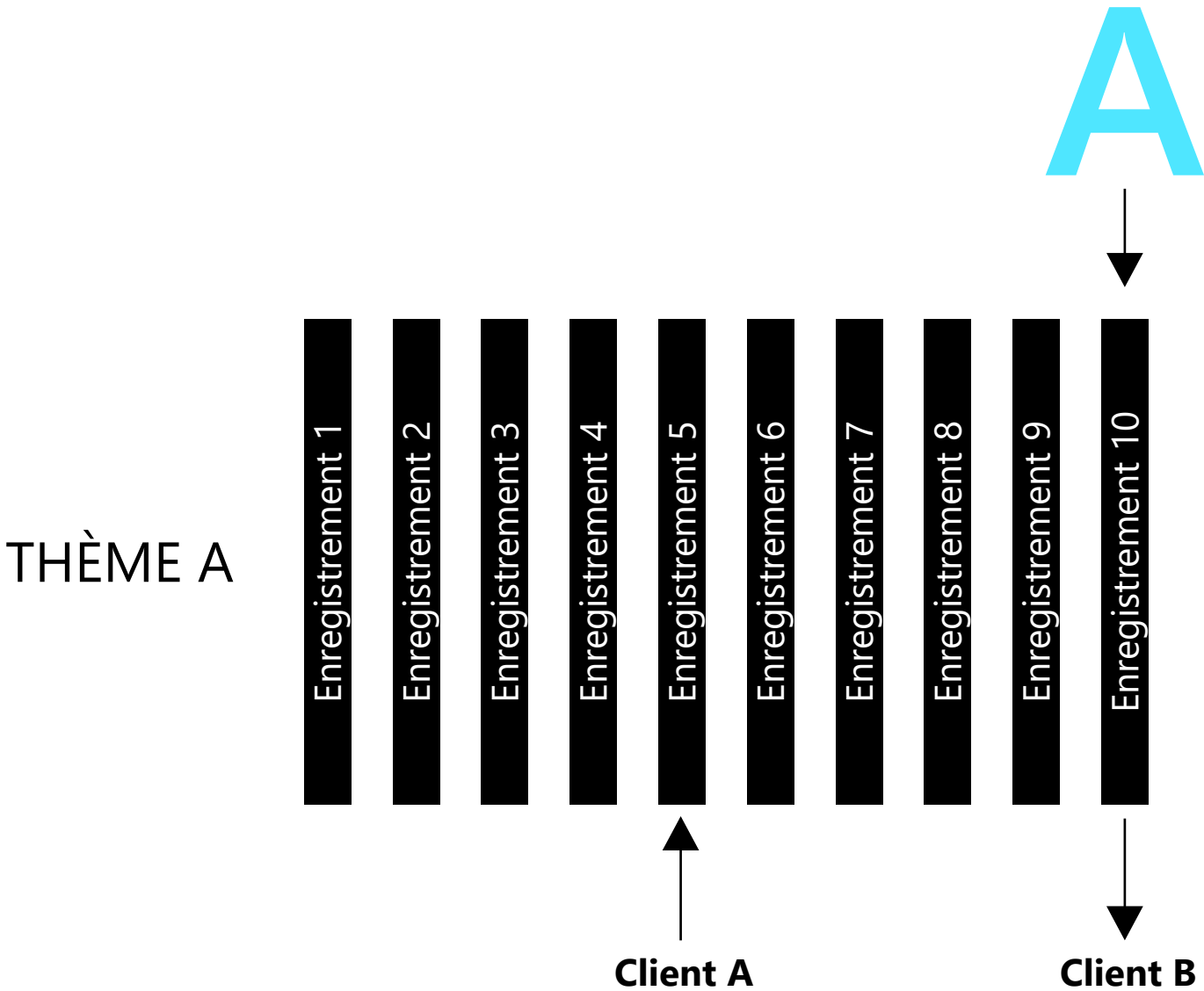
Les professionnels qui maîtrisent les architectures pilotées par les événements (EDA) mesurent le potentiel des événements dans l'intégration des services et le dimensionnement des systèmes. L'exposition de certains événements internes ou l'émission d'événements rudimentaires dans le cadre d'une plateforme d'API présente deux avantages : cela limite le couplage entre systèmes et crée des opportunités d'innovation en toute agilité.

Les plateformes proposent des flux de données

La diffusion de données en continu assure la transmission ordonnée de messages atomiques qui entraînent une modification de leur état. Contrairement aux API web (qui fournissent des données et spécifient des comportements), le streaming se concentre sur les données brutes.

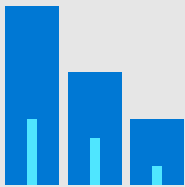
En transmettant des messages transactionnels entre les systèmes, les courtiers classiques constituent le réseau dorsal de bon nombre d'entreprises. Les flux de données diffèrent des courtiers de messages classiques en cela qu'ils privilégient les flux de messages inaltérables prenant en charge uniquement les ajouts (aux dépens des transactions). Cette évolution prend en charge le débit élevé et permet même de consulter l'historique des messages. Les clients peuvent relire des messages anciens par ordre de publication à mesure que de nouveaux besoins émergent ou que des mesures correctives sont nécessaires.

Les flux de messages viennent d'être ajoutés à la plateforme d'API. Ils permettent de mettre en pratique la gouvernance des données et de respecter les exigences de traçabilité dans les secteurs réglementés. En outre, cela devient la méthode de prédilection pour les solutions d'analyse des données et de Machine Learning qui doivent agir sur les données à mesure qu'elles sont disponibles, contrairement à l'approche plus traditionnelle de traitement des données par lots avec des technologies comme Hadoop.



Les flux de données permettent aux clients de traiter des messages nouveaux et anciens en vue de leur analyse.

Solutions	Capacités	Opérations	Gestion des données	Traitement des données
Application web d'achat	Rechercher des produits	Requête sur les produits disponibles	BD des produits disponibles	Analyse des produits les plus vendus
	Afficher les informations sur un produit	Requête sur les informations du catalogue de produits	BD des catalogues de produits	
API de marché tierce	Passer une commande	Commande pour passer une commande	BD des commandes	Moteur de recommandation de produits
	Afficher l'état d'une commande	Événement de commande passée	Flux de l'historique de navigation	
	Expédier une commande	Requête sur les niveaux de stocks	BD des produits recommandés	
Compétence d'achat Alexa (voix)	Mettre à jour la description d'un produit	Requête sur les produits recommandés	—	
	Mettre à jour les niveaux de stocks	Événement de produit commandé	—	



L'une des principales priorités d'une plateforme d'API en développement est d'agir sur les données en temps réel. En faisant évoluer le mode d'utilisation de vos propres données, vous serez en mesure de les analyser et de gagner en agilité.

La gestion des données d'entreprise alimente les API en diffusant des messages et des données.

Architecture de services et d'applications en maillage



L'architecture MASA désigne un nouveau modèle développé par Gartner. Elle reflète l'évolution observée au cours des dernières années, qui consiste à passer des applications à la numérisation de fonctionnalités et à la conception de plateformes d'API. L'architecture MASA permet d'établir des connexions dynamiques entre les services, les personnes et les processus sur l'ensemble des canaux et des appareils. Elle intègre la plateforme d'API avec des API externes dans de nouvelles solutions, ce qui permet aux entreprises de réagir bien plus vite aux changements du marché, comparé à ce qui se révélait possible avec des applications cloisonnées.

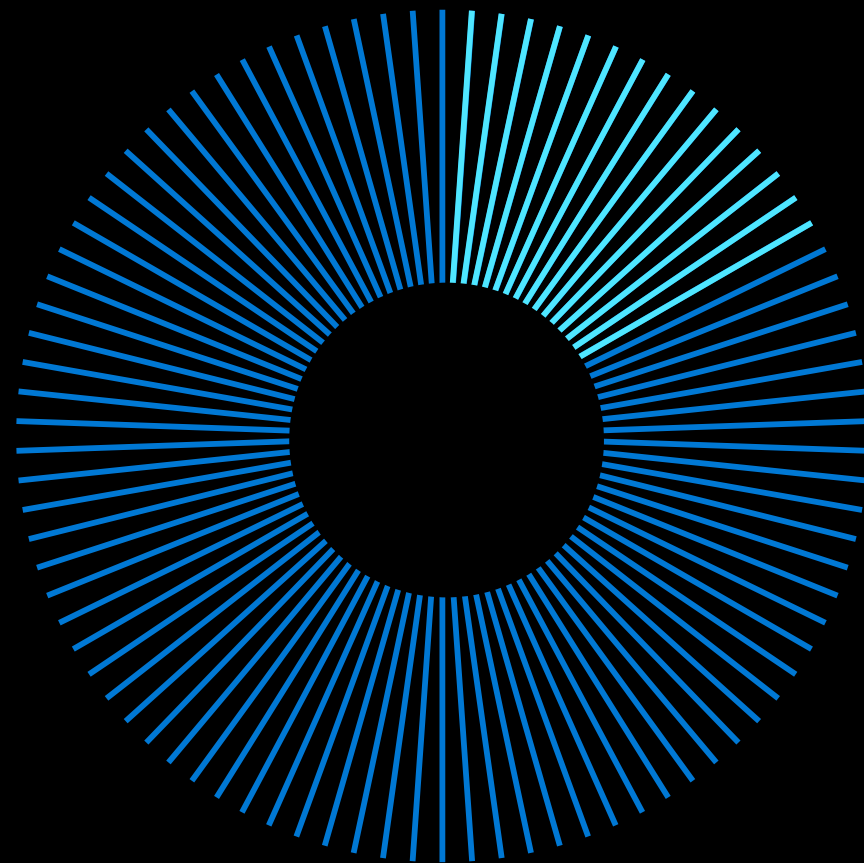
Voici l'élément clé de l'architecture MASA : la plateforme d'intégration en tant que service (iPaaS). Une plateforme iPaaS permet d'intégrer des API et des événements dans des solutions d'intégration qui connectent des systèmes, ce qui a pour effet de générer d'autres capacités. Ces fonctionnalités peuvent être déployées en tant que services ou sous forme de nouvelles API qui pourront être utilisées par d'autres applications.

Avec l'essor de l'architecture MASA et de la plateforme iPaaS, les développeurs peuvent désormais concevoir des applications avec des exigences limitées en matière de code. Cela permet aux développeurs de logiciels de se concentrer sur les tâches les plus difficiles de création de fonctionnalités numériques en tant qu'API. Quant aux utilisateurs professionnels qui maîtrisent des éléments comme les scripts Excel dans Visual Basic pour Applications (VBA), ils sont en mesure de créer des applications complètes sans faire appel au service informatique, souvent à court de ressources.

Un service marketing peut même développer des applications à durée de vie limitée (dans le cadre d'une conférence, par exemple), qui sont mises hors service dès qu'elles ne sont plus nécessaires. Étant donné le coût élevé du développement d'applications éphémères jusqu'à présent, cela n'était pas envisageable. Nous n'en sommes qu'au début, mais cette option devient possible.

CHAPITRE

4



L'ÉMERGENCE DES MICROSERVICES

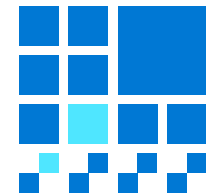
Le retour des logiciels modulaires

Ces dix dernières années, nous avons vu émerger de nouvelles infrastructures de développement visant à faciliter la conception et le déploiement d'applications web. L'inconvénient, c'est que ces infrastructures encouragent rarement la conception modulaire. Ainsi, le développement est rapide au démarrage d'un nouveau projet. Mais au fil du temps, la complexité augmente considérablement, dépassant même la capacité de l'infrastructure à garantir l'efficacité qu'elle offrait au départ.

Dans le même temps, des outils de conteneurisation comme Docker et des couches d'orchestration comme Kubernetes sont apparus. Ces outils incitent les développeurs à scinder les applications complexes en composants plus maniables, puis à les déployer de manière prévisible.

Dans ces circonstances, la conception de logiciels modulaires suscite un regain d'intérêt. Auparavant, on suivait une technique de développement basé sur les composants. Aujourd'hui, on conçoit des modules en tant que services avant de les externaliser sur le réseau. Cela entraîne l'extension des microservices.

Qu'est-ce qu'un microservice ?



Comme de nombreuses notions informatiques, les « microservices » évoquent toutes sortes de choses. Certains considèrent les microservices comme de petits bouts de code qui forment une API web. D'autres voient en eux l'espoir d'accélérer le développement pour leurs équipes en difficulté. Ces deux perceptions sont vraies, mais les microservices vont bien au-delà de ce qu'on imagine.

Un microservice désigne une unité de code autonome pouvant être déployée de manière indépendante dans le but d'effectuer correctement une tâche précise. Contrairement à ce que pensent certains, les microservices ne sont pas limités à un nombre de lignes spécifique.

Une architecture de microservices présente les caractéristiques suivantes :

Applique

des contextes délimités pour restreindre la charge cognitive.

Se déploie

de façon indépendante via l'automatisation CI/CD.

Facilite

la substituabilité et l'expérimentation en limitant la portée de chaque service.

Encourage

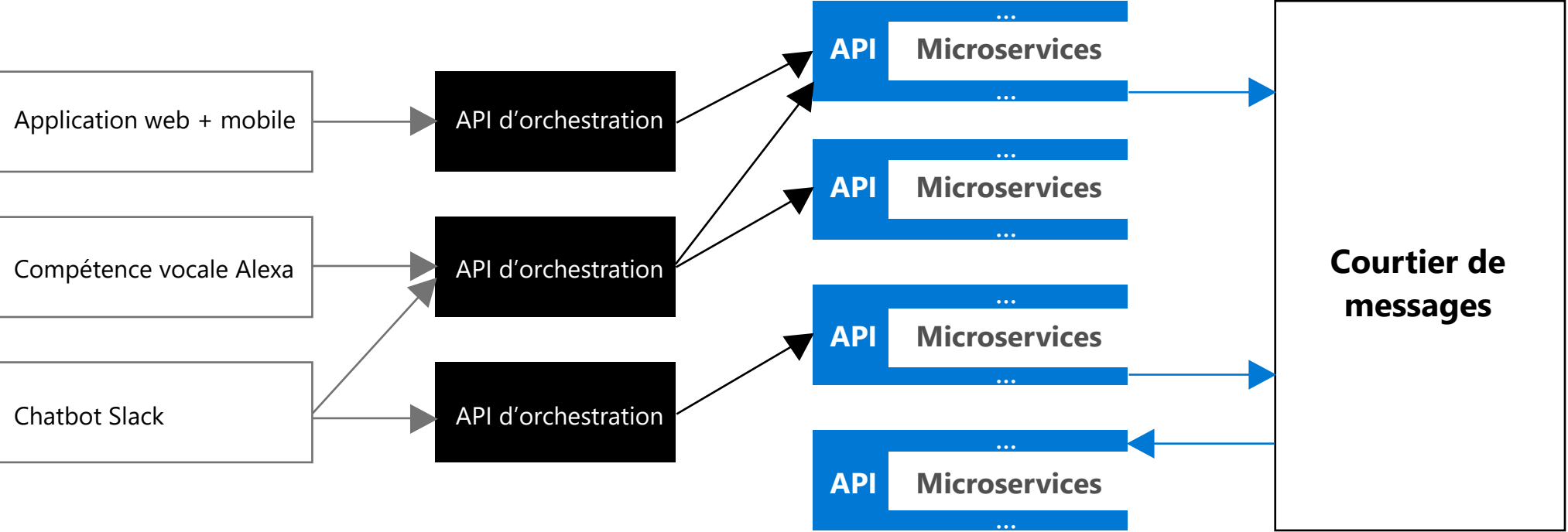
le dimensionnement optimal grâce au fractionnement continu et à la recombinaison des services.

Gère

ses propres sources de données et ne les partage pas avec d'autres services.

L'architecture de microservices

Elle se compose de services basés sur des API et de services pilotés par des événements ou des messages. On inclut généralement des API d'orchestration frontale qui font appel à un ou plusieurs microservices. Les entreprises peuvent ainsi proposer des contrats solides via les API d'orchestration, tout en remplaçant ou en fractionnant les microservices en arrière-plan. Ces opérations n'ont aucun impact sur les utilisateurs d'API.



Architecture de microservices courante utilisant des API d'orchestration qui font appel à un ou plusieurs microservices en arrière-plan.



Une entreprise qui s'engage sur la voie des microservices ne réalisera peut-être pas tout de suite les avantages offerts. Par conséquent, les entreprises qui mettent le cap sur les microservices doivent prévoir une marge pour faire mûrir leur approche en amont.



Les avantages d'une architecture de microservices

Le souhait de toutes les entreprises est de raccourcir les délais de conception des logiciels. Elles doivent toutefois garantir que leurs logiciels fonctionnent comme prévu, de façon homogène et précise. Par conséquent, des processus sont mis en place pour limiter les risques en voulant aller trop vite. Cela inclut souvent des réunions visant à coordonner le travail de l'ensemble des équipes. Chaque fois qu'il faut modifier le fonctionnement d'une application ou d'un service, des réunions de coordination doivent être organisées. Cela ralentit les choses.

Il est toujours difficile de faire un choix entre vitesse et sécurité. Les entreprises doivent trouver le juste équilibre pour concevoir des logiciels aussi rapidement que possible et atténuer les risques d'introduire des bogues qui affecteront la sécurité et la stabilité des interactions entre les clients et les partenaires.

Une architecture de microservices a pour premier objectif de réduire les coûts de coordination de plusieurs équipes/développeurs qui travaillent sur un système complexe. Pour ce faire, il convient de limiter la portée d'un service spécifique, tout en garantissant son déploiement indépendant. Quand il faut apporter une amélioration ou une correction, l'opération est effectuée rapidement à l'intérieur du microservice avant son déploiement en toute confiance. Les sources de données sont associées à un seul et même service. Elles ne peuvent pas être partagées entre différents services. Autrement, les coûts de coordination des microservices seraient plus élevés si des modifications étaient appliquées à un magasin de données partagé.

Les microservices ne sont pas la panacée

Certes, les microservices sont de plus en plus populaires, mais ils ne sont pas forcément adaptés à toutes les entreprises. À l'origine, les microservices étaient destinés à scinder des systèmes très sophistiqués en diluant leur complexité dans l'ensemble de l'entreprise, plutôt que de la concentrer dans un seul domaine. Ils sont donc parfaits pour les grandes entreprises qui emploient un nombre élevé de développeurs. Les PME possédant des systèmes de complexité moindre trouvent souvent que le surcroît de travail requis par la gestion de services indépendants l'emporte sur les avantages d'une architecture de microservices.

Autres éléments à prendre en compte :

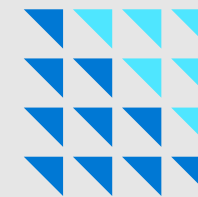
Les microservices requièrent un système entièrement automatisé. Un développeur doit être en mesure de concevoir un microservice, d'enregistrer le service auprès d'un système de déploiement automatisé, puis de déployer le code en fonction des besoins, sans approbation ni processus manuels. Les entreprises qui n'ont pas totalement automatisé leur pipeline de déploiement et supprimé la totalité ou la plupart des processus manuels risquent de rencontrer de gros problèmes. Il leur faudra en effet utiliser les microservices existants pour déployer de nouvelles fonctionnalités, ce qui générera des silos.

Les microservices doivent pouvoir être déployés de façon indépendante. Certaines entreprises choisissent de suivre leurs processus existants, ce qui conduit au déploiement coordonné de tous les microservices à la fois. Elles créent ainsi un monolithe distribué qui les empêche d'exploiter pleinement le potentiel de vitesse et de sécurité des microservices.

Les microservices doivent être détenus, surveillés et gérés par une seule équipe. Une équipe peut posséder quelques microservices, mais elle ne doit pas être responsable d'un grand nombre de services. Parfois, la structure hiérarchique et la culture d'entreprise sont en contradiction avec cette répartition de la propriété. Il est alors difficile de déployer correctement une architecture de microservices complète qui allie la vitesse et la sécurité escomptées. Prenez ce point en considération avant de passer aux microservices.

Les sources de données seront distribuées car chaque service doit posséder ses données. Cela nécessite de réaliser un investissement important dans la gestion des données distribuées via des processus ETL ou de diffusion pour regrouper les données issues de plusieurs services à des fins de génération de rapports. Les entreprises qui possèdent des bases de données volumineuses doivent faire preuve de prudence lors de la transition vers une architecture de microservices.

La voie vers les microservices requiert une parfaite connaissance des systèmes distribués. Les personnes qui ne maîtrisent pas les concepts de traçage distribué, d'observabilité, de cohérence éventuelle et de sagas distribuées passeront par des moments difficiles pour mettre en œuvre les microservices.



En général, les entreprises doivent procéder par étapes. Il faut essayer de décomposer des solutions en petites unités de code déployé de manière indépendante. Il arrive toutefois que l'adoption d'une architecture de microservices ne soit pas la bonne solution. À la place, il suffit parfois de décomposer les systèmes internes en monolithes modulaires avec des contextes clairement délimités qui exposent des API web et des événements métier pour répondre aux besoins de l'entreprise.

CHAPITRE

5

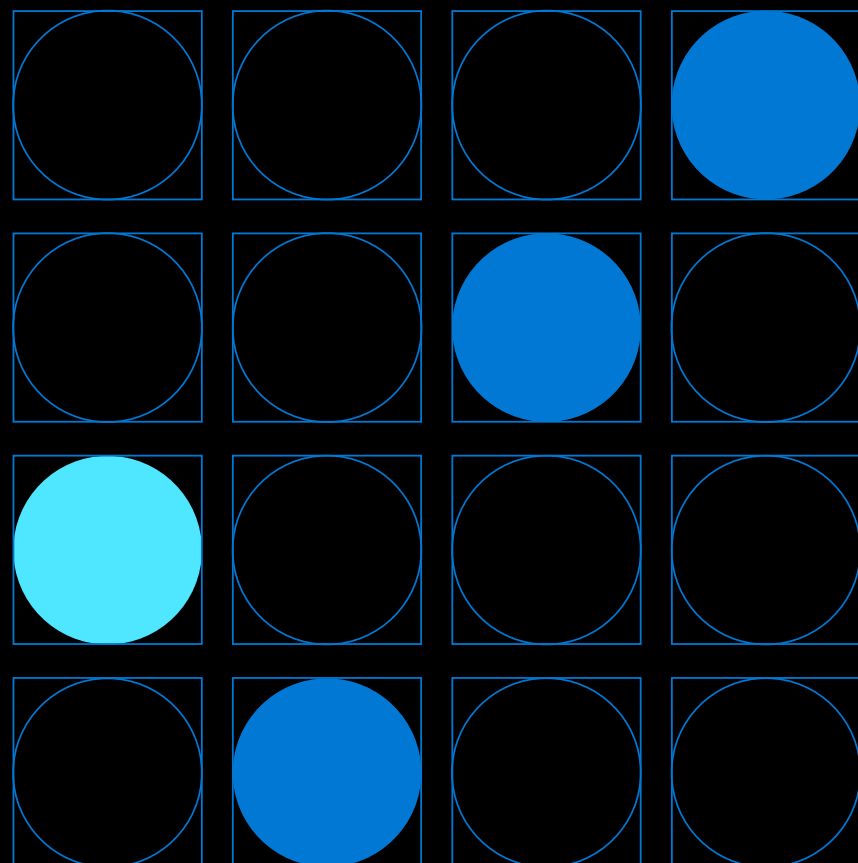
LES COMPOSANTS D'UNE PLATEFORME D'API

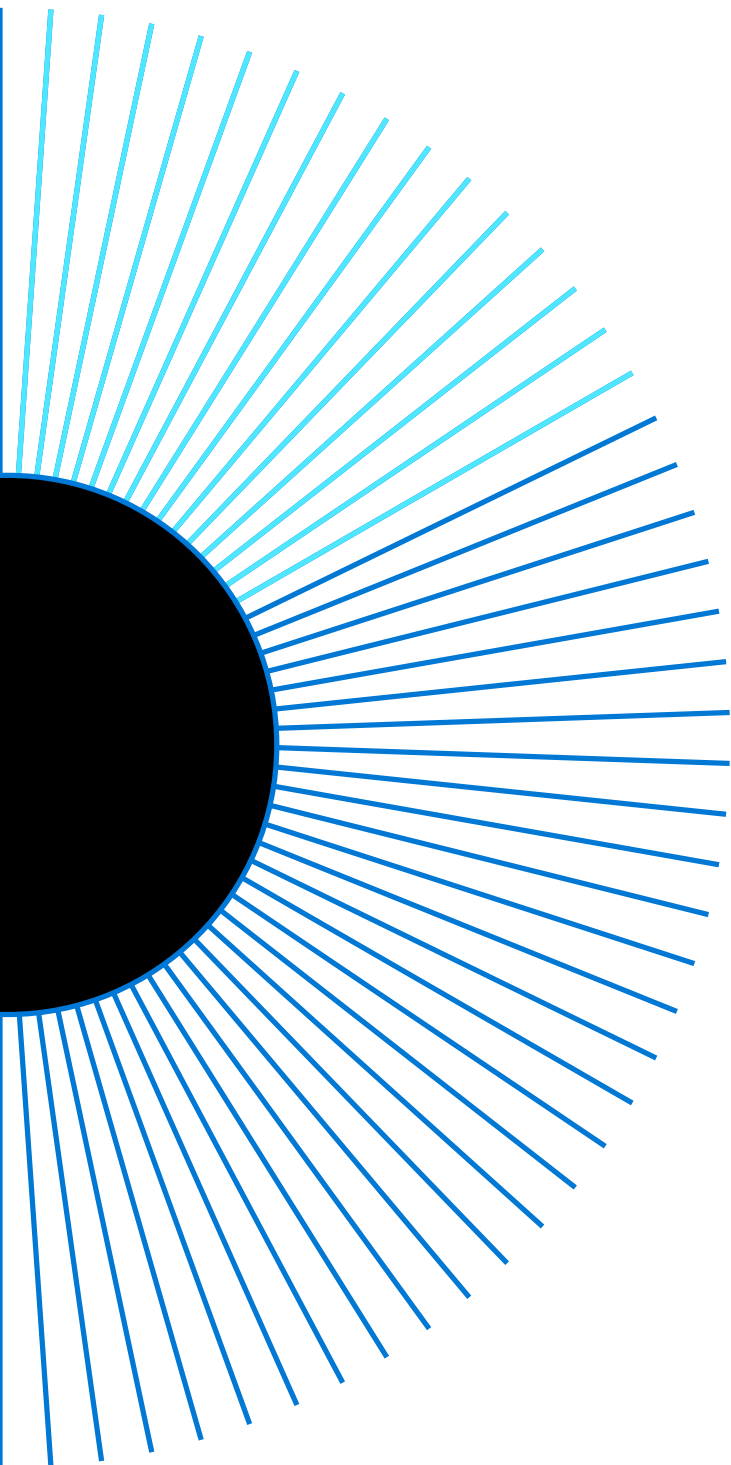
Gestion et sécurité des API

Les couches de gestion des API (APIM) accélèrent le déploiement, la surveillance, la sécurité, la gestion des versions et le partage des API. Elles sont souvent déployées sous forme de proxy inverse, qui intercepte toutes les demandes d'API entrantes et applique des stratégies pour déterminer si l'acheminement de ces requêtes est autorisé. La couche APIM peut être hébergée entièrement dans le cloud (comme c'est le cas avec la [gestion des API Azure](#) ou la [passerelle API open source Tyk](#)) ou sur site, mais les entreprises peuvent également adopter une approche hybride.

En outre, la couche APIM applique des limites de débit pour empêcher l'utilisation illimitée des API, renforce la sécurité via les jetons d'API et/ou OAuth 2 et neutralise un certain nombre de vecteurs d'attaque. Certaines entreprises proposent la gestion des identités en tant que fonction intégrée, mais la plupart configurent la couche APIM pour travailler avec un fournisseur d'identité tiers en vue de se délester de la gestion et de l'authentification des utilisateurs.

N'oubliez pas que la sécurité est un processus constant (et non un produit). Même si les menaces de sécurité évoluent en permanence, nous pouvons tout de même constituer un ensemble de bonnes pratiques et effectuer des tests. Pensez à mettre au point des techniques fiables pour détecter la moindre attaque malveillante contre votre surface d'exposition d'API.





Fournisseur d'identité

Un fournisseur d'identité (IdP) crée et gère les identités et les informations d'accès à votre API. Il propose également des services d'authentification à la couche APIM et prend en charge l'authentification unique (SSO) via des technologies comme le langage SAML (Security Assertion Markup Language). Certains IdP assurent la prise en charge de la couche OpenID Connect (OIDC), qui vient compléter OAuth 2.0 avec des autorisations supplémentaires à l'aide d'API web et de jetons d'identité JSON.

Réseau de distribution de contenu

Un réseau de distribution de contenu (CDN) désigne un cache distribué à la périphérie d'Internet qui permet de réduire la charge sur les serveurs d'origine. Étant donné que le contenu est souvent plus proche d'un appareil que des serveurs d'origine, cela permet de renvoyer plus rapidement les réponses d'API en cache vers des applications web ou mobiles. Un CDN offre également une protection supplémentaire contre les attaques par déni de service distribué (DDoS) en surveillant le trafic des couches 3 et 4, en détectant les éventuels problèmes et en empêchant ceux-ci de perturber les clients, les partenaires et les systèmes internes.



Automatisation du déploiement

Il est fondamental d'automatiser le processus de conception et de déploiement d'un programme d'API à forte valeur ajoutée. La plupart des entreprises effectuent des contrôles manuels et suivent des étapes de vérification dans le cadre de la mise en production, mais il est possible de raccourcir les délais en réduisant leur nombre. Pour ce faire, il faut mettre en place un pipeline CI/CD à toute épreuve.

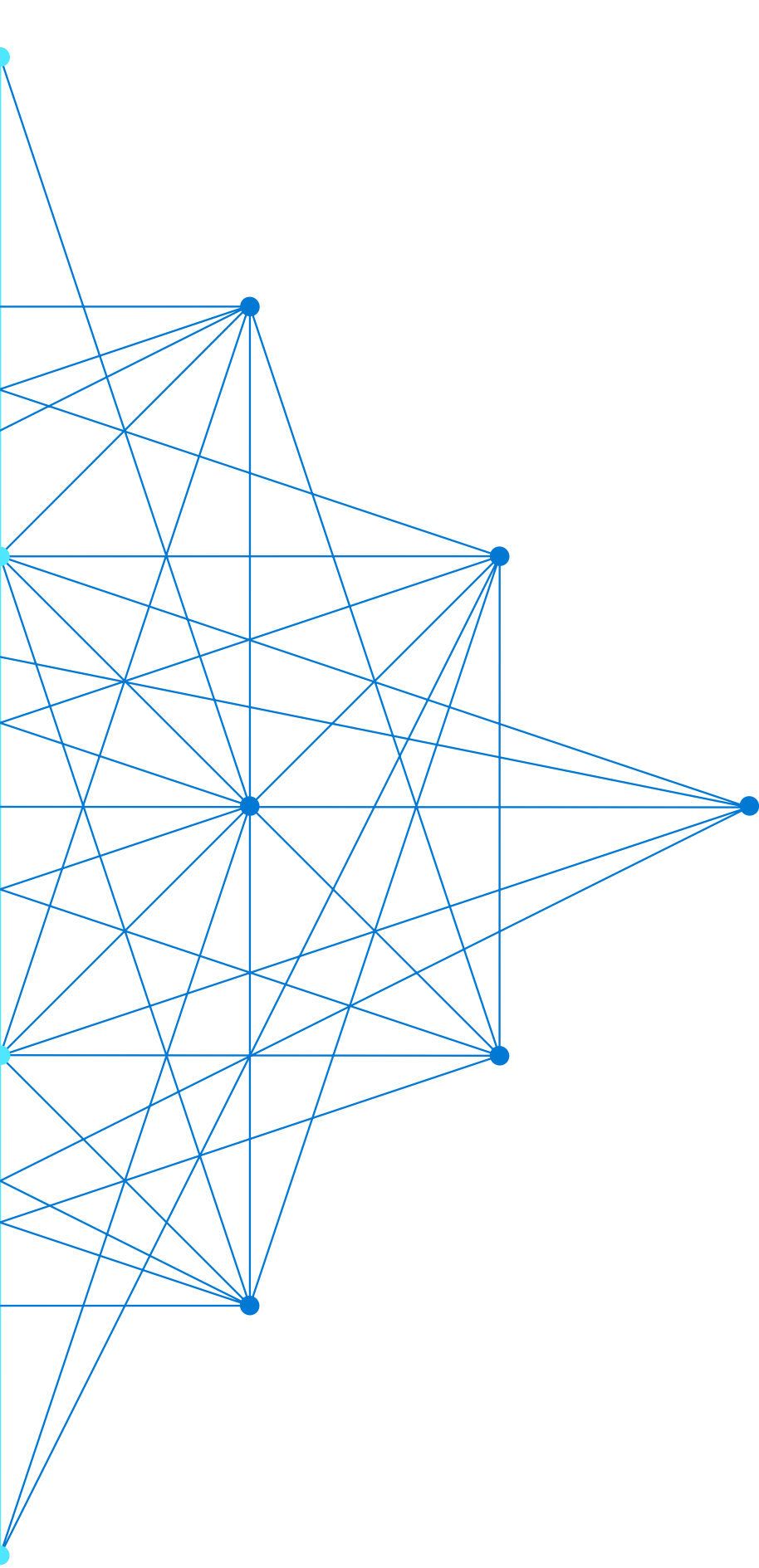
L'intégration continue (CI) impose aux développeurs de fusionner aussi souvent que possible les changements apportés avec la branche principale. Les modifications sont validées par la création d'un package de déploiement, puis l'exécution de tests automatisés. L'intégration continue privilégie les tests automatisés pour vérifier que l'API n'est pas endommagée à chaque nouvelle opération de fusion avec le tronc de code. Nous aborderons plus loin les stratégies de test automatisés des API.

La livraison continue (CD) vient compléter l'intégration continue en incluant l'automatisation des processus de mise en production via des scripts prévisibles. De nombreuses entreprises préfèrent encore un cycle de mise en production régulier (toutes les deux semaines, par exemple) et adoptent donc cette approche. Celles qui optent pour l'automatisation complète de la mise en production suivent une méthode de déploiement continu. Le déploiement continu prend en charge l'automatisation de la mise en production complète dès que le code est enregistré, validé et déployé.

Des solutions open source comme [Ansible](#), [Chef](#) et [Puppet](#), ainsi que des solutions managées comme [Azure DevOps Services](#) fournissent tous les composants nécessaires pour répondre à vos besoins en matière de pipeline CI/CD, y compris des outils agiles, des référentiels Git et des fonctionnalités de pipeline configurables.

```
1 swagger: '2.0'
2 info:
3   title: Lab 1
4   version: '1.0'
5   host: example.com
6   paths:
7     /projects:
8       get:
9         summary: List Project
10        operationId: findProjects
11        responses:
12          '200':
13            description: ''
14            schema:
15              type: array
16              items:
17                $ref: '#/definitions/project-summary'
18        post:
19          summary: Create Project
20          operationId: createProject
21          parameters:
22            - in: body
23              name: body
24              schema:
25                $ref: '#/definitions/new-modify-project'
26          responses:
27            '201':
28              description: ''
29              schema:
30                $ref: '#/definitions/project-details'
31        '/projects/{projectId}':
32          parameters:
33            - name: projectId
34              in: path
35              type: string
36              required: true
37          get:
38            summary: Get Project
39            operationId: getProject
40            responses:
41              '200':
```

Exemple de définition d'API utilisant OpenAPI v2



Définition et documentation des API

Une bonne communication est un facteur essentiel à l'adoption des API. En fait, la documentation est la principale méthode pour expliquer aux développeurs comment utiliser une API. La documentation des API représente l'interface utilisateur de vos API.

Auparavant, la documentation des API était capturée dans un document statique au format HTML ou PDF, mais d'autres options sont à présent disponibles. Des outils comme Swagger, RAML et Blueprint ne sont que quelques exemples de formats disponibles pour capturer la définition d'une API dans un format lisible par une machine. Ces formats prennent en charge la génération de la documentation de référence d'une API.

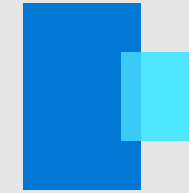
La plupart des entreprises profitent également de l'intégration de ces définitions dans le cycle de vie du développement de leurs logiciels (SDLC). Cela contribue à l'automatisation et à la configuration de la couche APIM et d'autres infrastructures.

Le portail des développeurs

La documentation complète des API nécessite plus que des documents de référence relatifs aux API. Un portail destiné aux développeurs regroupe les différents styles de communication dont vous avez besoin pour présenter vos API et les avantages liés à leur utilisation. Il offre des conseils aux développeurs pour bien démarrer avec l'intégration de vos API.

Un portail robuste inclut les sections suivantes :

1. **Présentation** pour décrire les cas d'utilisation de l'API et fournir des informations sur la connectivité de base.
 2. **Authentification et autorisation** pour préciser la méthode de sécurisation de l'API, expliquer comment obtenir un jeton d'accès et présenter des détails sur l'expiration des jetons.
 3. **Exemples de workflows**, généralement au format demande/réponse HTTP, y compris des exemples de code pour aider les développeurs à comprendre comment résoudre des problèmes courants avec l'API.
 4. **Informations sur la limitation du débit et les contrats SLA** pour expliquer comment l'utilisation de l'API par une application peut être limitée, et préciser les tarifs (le cas échéant).
 5. **Documentation de référence de l'API** habituellement générée à partir des définitions d'API, destinée aux développeurs qui doivent appréhender chaque point de terminaison de l'API.
-

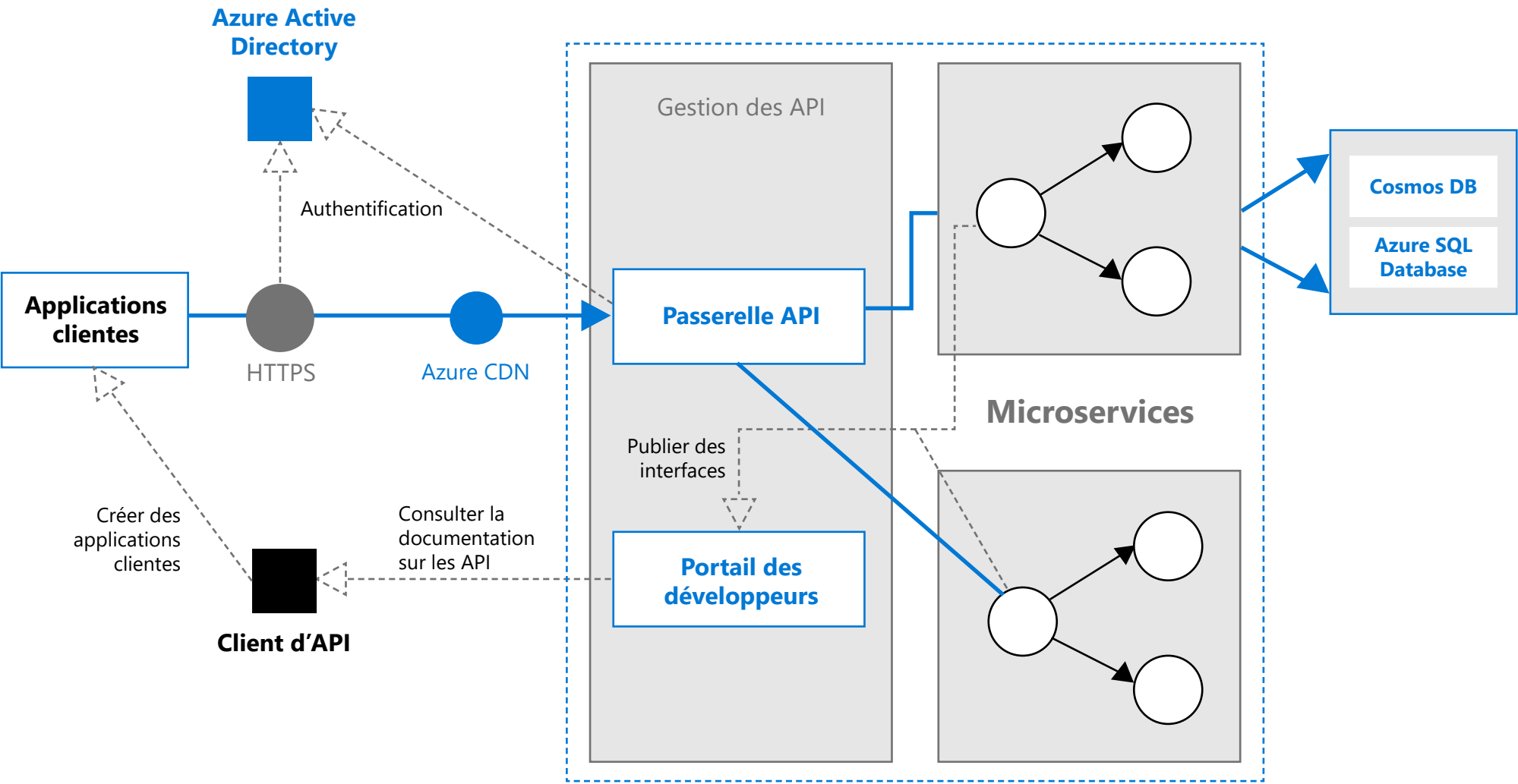


Cette liste peut sembler rebutante, mais il est conseillé d'insérer un minimum de contenu dans chaque section pour répondre aux besoins courants des utilisateurs cibles. Vous pouvez ensuite étoffer le contenu au fil du temps jusqu'à ce que le portail des développeurs aborde la plupart des questions et des cas d'utilisation courants associés à votre API.

Certains fournisseurs de couches APIM open source et commerciales prennent en charge la publication de vos documents sur un portail, ce qui permet aux développeurs de disposer de la documentation essentielle.

Schéma synthétique :
architecture d'API moderne

Le schéma ci-dessous illustre les composants mentionnés travaillant ensemble pour prendre en charge les applications clientes à l'aide d'API et de microservices.



Architecture d'API et de microservices reposant sur Azure Cloud Services

CHAPITRE

6

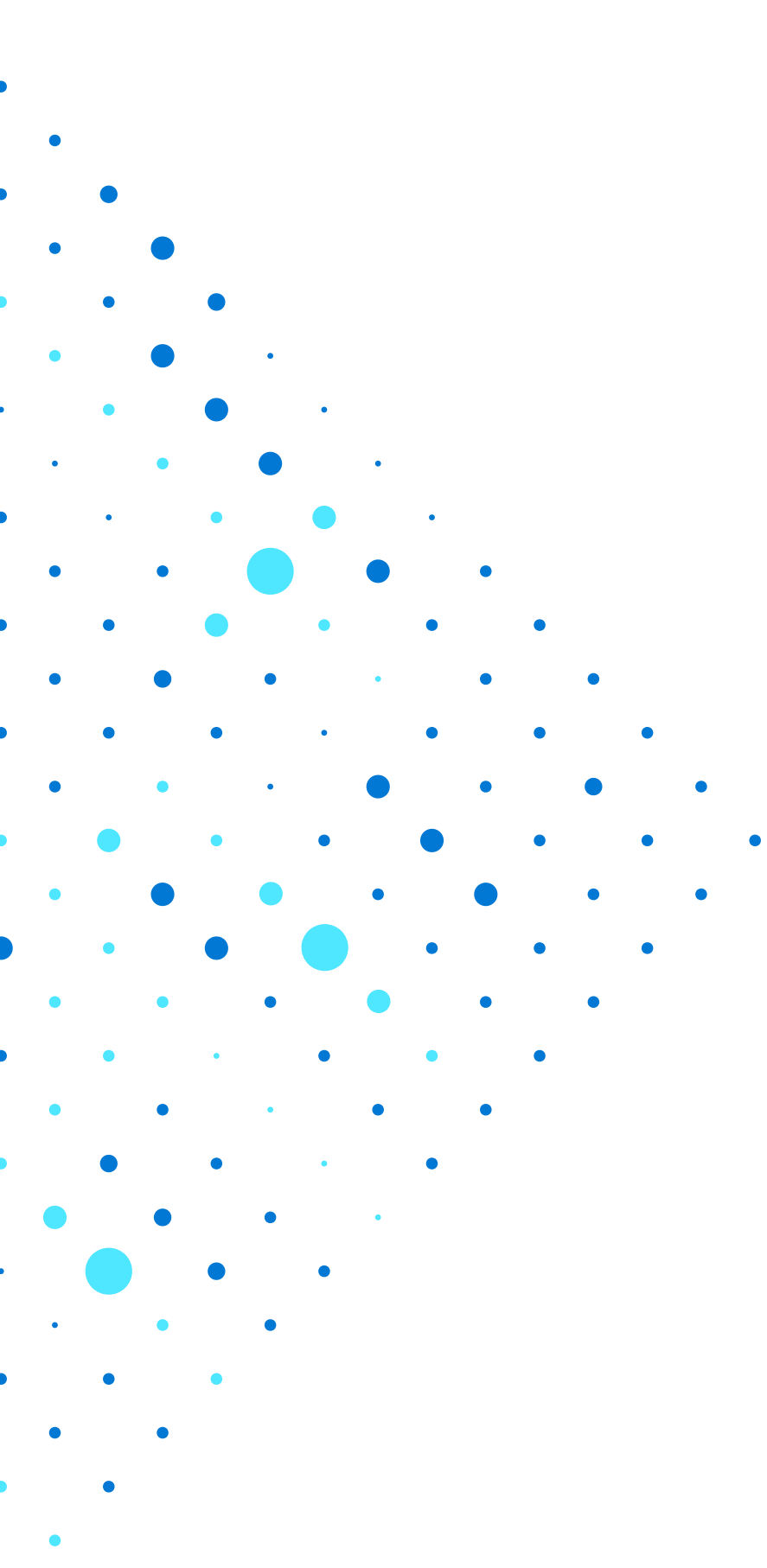
PLANIFIER L'ÉVOLUTION VERS UNE ACTIVITÉ CENTRÉE SUR LES API

Élaborer votre stratégie d'API

Les API vont au-delà d'un ensemble de technologies présentant un intérêt pour les développeurs. Elles améliorent l'expérience client, renforcent les relations commerciales et stimulent l'innovation interne. Une stratégie d'API précise transforme vos données et vos processus en fonctionnalités numériques avec une interface programmable.

Malheureusement, certaines entreprises se rendent compte trop tard que leurs équipes sont déjà en train de concevoir et de déployer des API. Ces API sont souvent créées de manière isolée pour résoudre un problème spécifique et ne présentent pas la cohérence nécessaire pour être réutilisées.

Si vous définissez clairement les objectifs du programme d'API de votre entreprise, vos équipes feront en sorte d'apporter la valeur ajoutée indispensable pour les atteindre. Après avoir fixé les objectifs de votre programme d'API, documentez-les précisément et partagez-les avec l'ensemble des employés. Utilisez des indicateurs de performances clés (KPI) pour évaluer la réussite de votre programme, effectuez un suivi à long terme et mettez vos opérations en adéquation avec les objectifs à atteindre.



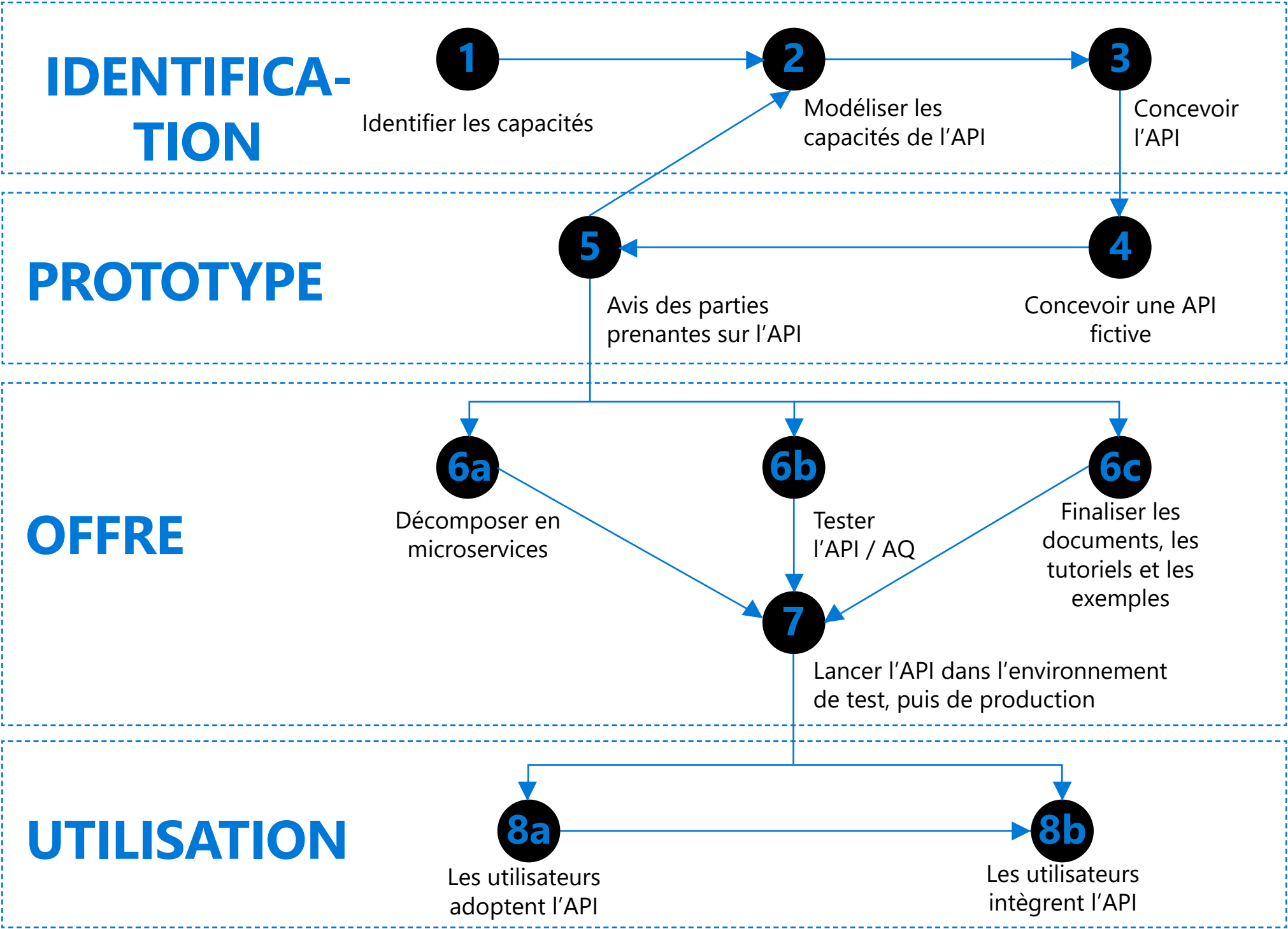
Établir un processus donnant la priorité à la conception d'API

La conception d'API est trop souvent ignorée ou précipitée. Il est donc impossible de saisir l'opportunité d'approfondir et d'améliorer la conception d'une API avant sa mise en production. En adoptant une approche donnant la priorité à la conception, on peut incorporer la rétroaction avant la mise en production d'une API. Cela évite d'apporter des changements importants juste après son lancement.

Dans le cadre d'une stratégie donnant la priorité à la conception, on commence par identifier les capacités souhaitées, puis on s'oriente vers la création d'une API pour fournir ces fonctionnalités. Toutes ces opérations ont lieu avant d'écrire la moindre ligne de code. Cela peut s'apparenter à une approche séquentielle, mais ce n'est pas tout à fait identique. C'est plutôt la mise en pratique des principes d'agilité, qui visent à associer des éléments métier et technologiques sous forme de petites itérations qui apportent une valeur ajoutée.

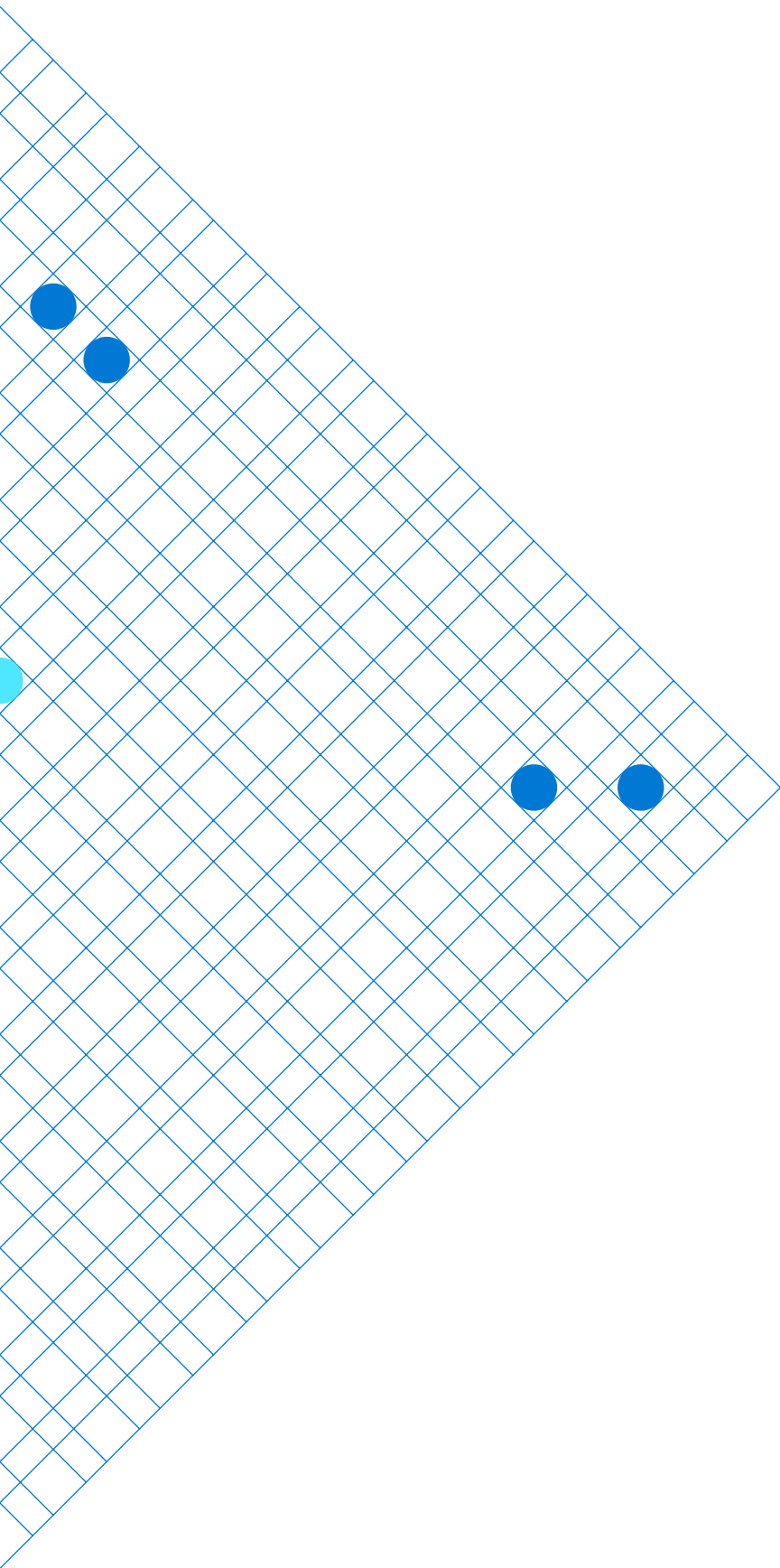


Le processus englobe les étapes suivantes, exécutées en parallèle dans la mesure du possible :



Un processus donnant la priorité à la conception d'API tient compte de l'avis des parties prenantes.

- 1. Identifier les fonctionnalités à fournir.** Concentrez-vous en premier lieu sur le comportement et les résultats escomptés avant de vous intéresser au code.
- 2. Modéliser et concevoir l'API.** Transformez les fonctionnalités souhaitées en approche basée sur les ressources grâce au potentiel de HTTP.
- 3. Documenter la conception de l'API initiale.** La documentation doit être suffisamment précise pour exposer les cas d'utilisation courants.
- 4. Demander l'avis des parties prenantes.** Basez-vous sur leurs commentaires pour confirmer que la conception de l'API répond aux besoins des intervenants.
- 5. Mettre à jour la conception de l'API.** Le cas échéant, incorporez la rétroaction dans la conception avant que le code ne soit mis à jour. Répétez les étapes 1 à 4 aussi souvent que nécessaire pour affiner la conception de l'API.
- 6. Produire une version fictive de l'API.** À mesure que la conception prend forme, demandez aux parties prenantes d'examiner la définition capturée (généralement au format OpenAPI).
- 7. Lancer l'API implémentée en tant que version préliminaire.** Au cours de l'évaluation de cette version préliminaire, des informations exploitables et des défauts de conception seront mis à en lumière. Apportez les changements nécessaires avant la mise en production officielle de l'API.
- 8. Engager la mise en production.** Dès que la version préliminaire répond aux besoins des parties prenantes, lancez l'API dans l'environnement de production. À ce stade, aucune modification importante n'est autorisée.



N'oubliez pas qu'il est possible de suivre ce processus progressivement pour fournir des points de terminaison en lecture seule, puis des points de terminaison qui manipulent les données ou prennent en charge davantage d'éléments du workflow. Profitez de la phase d'évaluation de la version préliminaire pour identifier les éventuelles anomalies, améliorer la documentation et développer des exemples de code illustrant l'utilisation de l'API.

Si vous privilégiez les échanges avec les parties prenantes et prenez leurs commentaires en considération, vous n'aurez pas à remanier et à réinstaller l'API dans des délais serrés, et vous apporterez une valeur ajoutée en continu.

Gérer votre portefeuille d'API

Certaines entreprises suivent des processus efficaces qui leur permettent de produire une grande quantité d'API et de microservices, mais elles n'organisent pas clairement leur portefeuille global. Un défaut de gestion du portefeuille d'une entreprise peut entraîner des doublons de fonctionnalités existantes par effet de spirale.

La gestion du portefeuille d'API inclut les étapes suivantes :

DIVISER

le portefeuille en plusieurs domaines. Les API ne sont pas toutes liées (comptes clients, commandes et stocks sur une plateforme de commerce électronique, par exemple). Classez vos API et vos microservices par catégories et attribuez la propriété du produit à chaque domaine.

ÉTABLIR

un processus précis pour ajouter de nouvelles API. En mettant en place une procédure claire pour intégrer des API et des microservices au portefeuille, vous éliminez tout risque de confusion quant à l'emplacement attribué des nouvelles API. Ainsi, les utilisateurs trouveront plus facilement les API dont ils ont besoin.

GÉRER

les chemins d'accès aux URL comme des noms de domaine. Les chemins d'URL sous le nom d'hôte de votre API (api.mycompany.com, par exemple) sont exactement comme des noms de domaine. C'est le patrimoine des développeurs. Gérez les chemins d'URL en utilisant des préfixes bien connus qui prennent en charge le schéma de partitionnement de votre domaine. Cela évite d'avoir des points de terminaison dispersés, des noms de ressources en double ou portant à confusion, ou encore des chemins contradictoires.



En assurant la bonne gestion du portefeuille d'API, toutes les équipes de l'entreprise peuvent contribuer aux offres globales. Il est également plus facile pour les équipes qui utilisent des API d'en chercher une qui répond aux besoins de leurs applications. Veillez à nommer un chef de produit ou à constituer une petite équipe qui sera propriétaire du portefeuille d'API (dans le cadre de votre stratégie de gouvernance des API, par exemple).

Définir le cycle de vie d'une API

Un programme d'API doit établir un cycle de vie précis et complet. Voici les sept étapes courantes du cycle de vie d'une API :

1. Identifier

Déterminez de nouvelles fonctionnalités d'API dès que les parties prenantes émettent des demandes.

2. Concevoir

Étudiez soigneusement les besoins avec les parties prenantes, identifiez les exigences en termes de capacités et concevez l'API en conséquence.

3. Fournir

Installez et testez les nouvelles fonctionnalités, puis vérifiez qu'elles répondent aux attentes des parties prenantes.

4. Mettre en production

Déployez, surveillez et gérez l'API lors de son exécution.

5. Partager

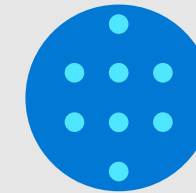
Présentez les nouvelles fonctionnalités aux parties prenantes et aux autres développeurs.

6. Améliorer

Incorporez la rétroaction en vous basant sur les commentaires émis, puis appliquez des techniques appropriées de gestion des versions pour optimiser l'API sans perturber les clients existants.

7. Mettre hors service

Informez les utilisateurs de la fin du support d'une API existante, orientez-les vers une nouvelle API (le cas échéant), puis mettez l'API hors service.



Intégrez les processus logiciels existants tout au long du cycle de vie de l'API, mais préparez-vous à les ajuster pour éviter tout effet négatif sur les utilisateurs des API actuelles.

Élaborer une stratégie de test d'API

Grâce à une stratégie de test adaptée, vous pouvez vérifier qu'une API fonctionne correctement et tient toutes ses promesses. Une stratégie de test d'API complète couvre le code, la fonctionnalité, le contrat et l'acceptation.

Tests unitaires

Il s'agit de savoir si le code sur lequel repose votre API fonctionne correctement. Ces tests permettent de reproduire un bogue identifié, de vérifier qu'il est corrigé et d'empêcher sa régression à mesure que le code continue à évoluer.

Tests fonctionnels

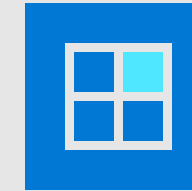
Ils vérifient que chaque point de terminaison de l'API a un comportement normal. Les tests fonctionnels doivent évaluer le comportement de chaque point de terminaison en cas de réussite ou d'erreur pour garantir que le code protège l'API face à tout code client incorrect ou malveillant.

Tests contractuels

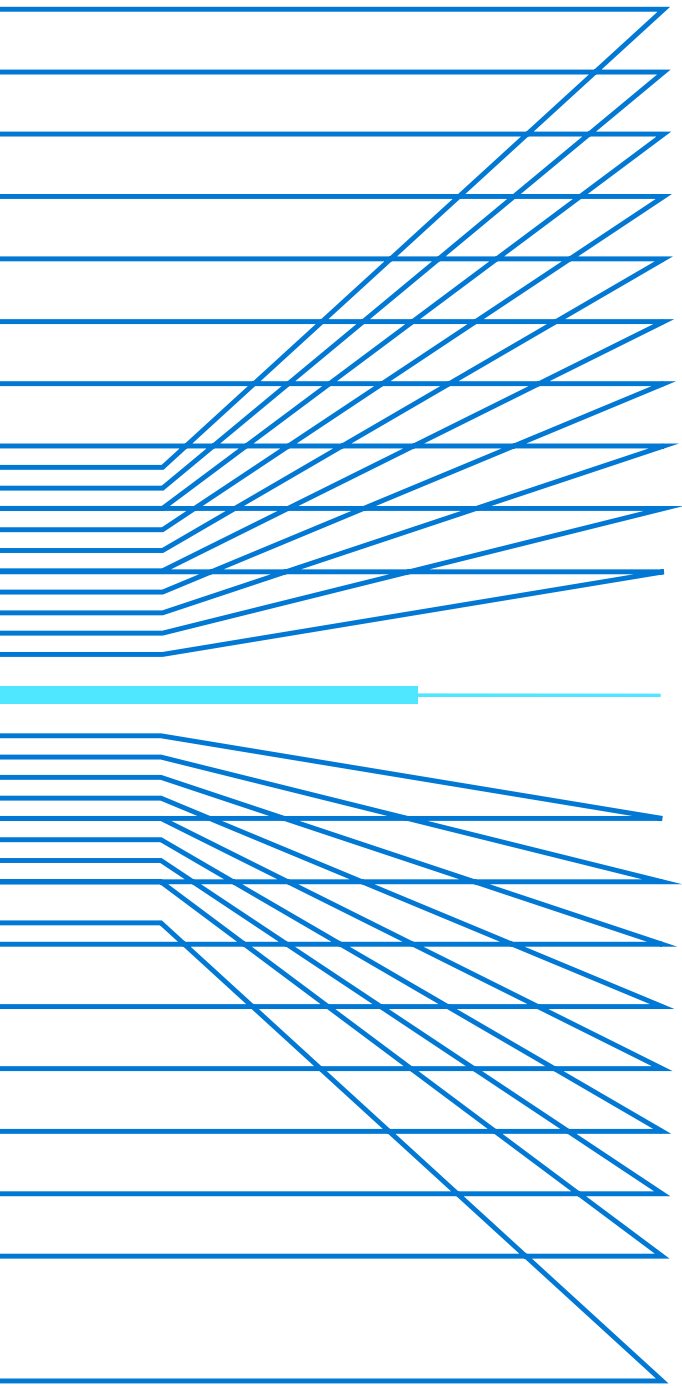
Ils vérifient que l'implémentation de l'API correspond à la spécification. Ces tests comparent le trafic de demande/réponse de l'API d'exécution aux définitions d'API (généralement au format OpenAPI, API Blueprint ou RAML) et signalent tout écart par rapport au contrat prévu.

Tests d'acceptation

On exécute une série de chemins prédéterminés qui représentent le mieux les types d'interactions que les utilisateurs auront avec votre API pour vérifier qu'un état final spécifique est atteint.



Pour chacun des aspects de votre stratégie de test, essayez de trouver des outils qui prennent en charge une approche donnant la priorité à la conception d'API. Si votre entreprise souhaite tester davantage le code, évaluez le code généré par les définitions d'API et effectuez des tests manuels, le cas échéant. Les équipes d'assurance qualité peuvent envisager de tirer parti d'outils de création automatique de suites de tests portant sur les définitions d'API. D'autres cas de test peuvent être définis par l'équipe AQ en fonction des besoins. Dans la mesure du possible, évitez de procéder à des tests manuels de votre API au-delà des tests exploratoires.



Surveiller vos API

Aujourd'hui, la surveillance des applications robustes ne se limite plus à vérifier que vos processus sont en cours d'exécution. Il s'agit plutôt de tracer une demande web entrante sur tous les points de terminaison et les services qui prennent en charge cette requête. Grâce à l'observabilité, vous pouvez savoir quand des déploiements récents génèrent un nombre accru d'erreurs, ce que le pipeline CI/CD n'aura peut-être pas détecté.

Les architectures modernes exigent généralement d'associer la journalisation distribuée, le traçage distribué et la surveillance des API (souvent offerte en tant que fonction au niveau de la couche APIM) pour évaluer les erreurs d'exécution et procéder au dépannage. Certaines entreprises trouvent que l'instrumentation disponible via un maillage des services constitue le complément idéal de ces outils lorsqu'elles adoptent les microservices.



Gouvernance de votre plateforme d'API

De nombreux programmes de gouvernance SOA historiques se concentraient en interne sur l'élimination des redondances opérationnelles. Les processus étaient donc lents et semblaient avoir peu d'intérêt pour les équipes qui produisent de nouveaux services. Les programmes d'API modernes sont passés à une approche orientée vers l'extérieur dont la priorité est d'apporter une valeur ajoutée.

Une initiative de gouvernance saine des API doit encourager la cohérence à l'échelle de l'entreprise, tout en garantissant la souplesse nécessaire pour s'adapter à l'évolution des exigences. Pour cela, il convient de réaliser les actions de gouvernance suivantes :

Encadrer

les employés et les familiariser avec les techniques de modélisation et de conception d'API pour constituer des équipes de production autonomes.

Produire

des supports de formation et d'autres ressources pour partager les connaissances.

Autonomiser

les équipes de solutions pour détecter et utiliser les API existantes dans le but d'optimiser la réutilisation d'une API au lieu de répéter les mêmes tâches.

Rédiger

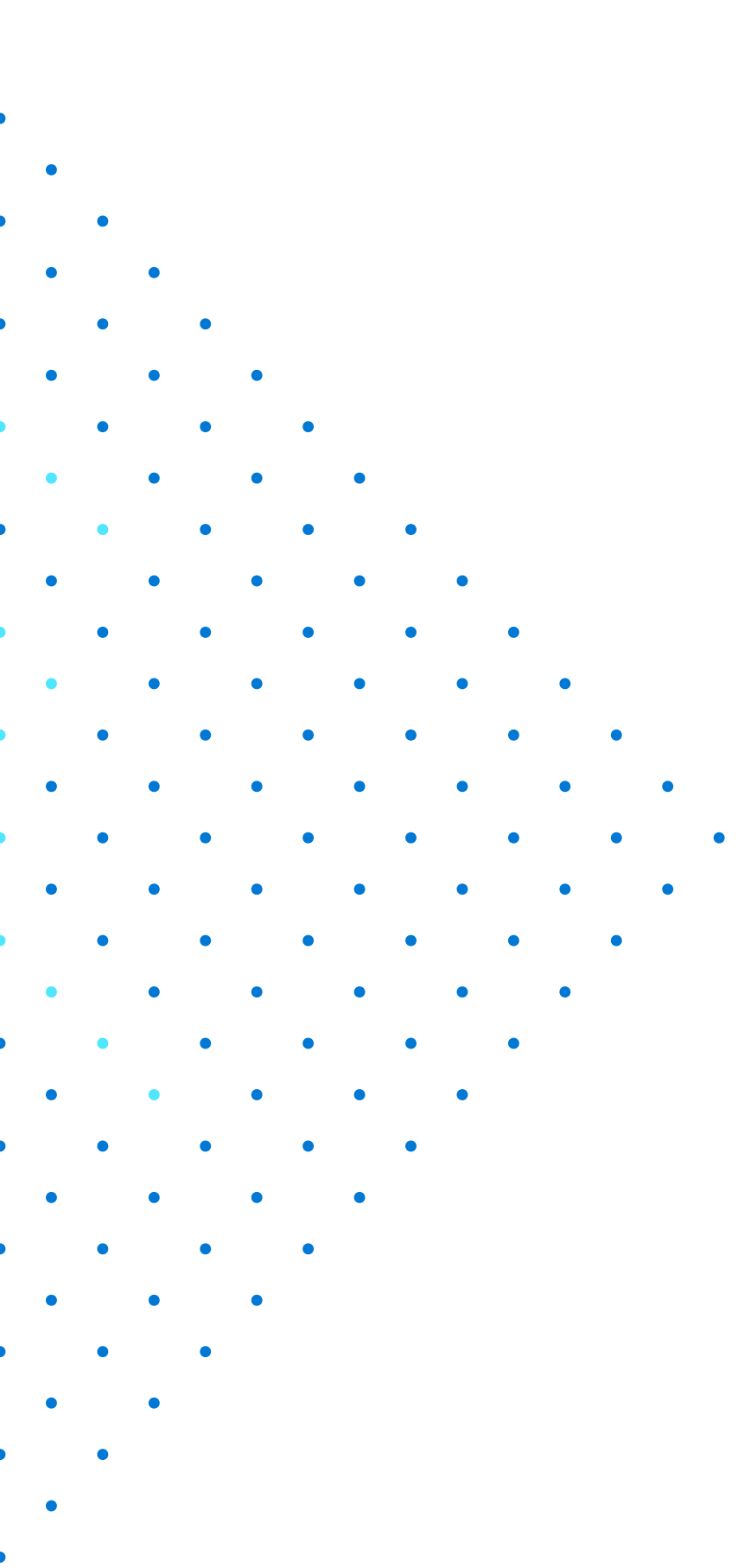
un guide de style précis avec des modèles de conception d'API qui prennent en charge les cas d'utilisation courants dans l'entreprise.

Élaborer

des stratégies d'intégration, de limitation du débit et de contrôle d'accès.

Établir

des processus et des pratiques flexibles qui encouragent l'innovation.



Les entreprises comptant un grand nombre d'équipes de développement ou des membres répartis sur plusieurs zones géographiques peuvent tirer parti d'un modèle de gouvernance fédérée. Dans ce modèle, une équipe centrale chargée de la gouvernance des API forme et autonomise les représentants de chaque unité commerciale ou région pour proposer des conseils en fonction du contexte et encadrer les équipes qui produisent des API. L'avis des représentants permet à l'équipe centrale de gouvernance des API d'ajuster les processus afin de mieux répondre aux besoins de l'ensemble des développeurs de l'entreprise.

La gouvernance des API peut être centralisée et gérée par une seule équipe tout au long de la durée de vie du programme d'API. Dans les grandes entreprises, la gouvernance peut être centralisée au début, puis évoluer vers une approche fédérée à long terme pour redimensionner le processus.



Intégrer des API existantes à votre plateforme

Vos API ne sont pas toutes développées dans le cadre d'un programme d'API établi. Il arrive que certaines soient conçues et déployées avant même la création du programme. D'autres peuvent être conçues en parallèle, avant qu'il y ait suffisamment de processus et d'employés dédiés. Préparez-vous, cela reflètera peut-être la réalité de votre programme d'API pendant un certain temps.



Une approche consiste à séparer ces API à l'aide de préfixes ou de noms d'hôte spécifiques (par exemple, `GET /legacy/accounts/1234` ou `https://legacy-api.mycompany.com/accounts/1234`). Cela définit un chemin clair pour les projets en cours, tout en isolant ces API de votre portefeuille d'API de base.

Pour les API déjà déployées sans couche APIM, commencez par identifier celles qu'il faut cataloguer et gérer en consultant les journaux de requêtes. Repérez les API à risque, collaborez avec l'équipe qui les a créées, puis établissez un plan pour les migrer derrière la couche APIM dès que possible.

Si certaines API ne sont pas associées à des définitions existantes, deux options s'offrent à vous. Soit vous pouvez utiliser des outils pour générer des définitions d'API à partir du code. Soit vous pouvez tirer parti d'outils de capture du trafic pour créer des fichiers d'archive HTTP (HAR) qui seront ensuite analysés par des outils capables de générer des définitions d'API à partir des sources d'exécution. Ces deux options courantes vous mettront sur la bonne voie en matière de documentation et de gouvernance.

CONCLUSION

Même s'il est parfois difficile de lancer et de respecter un programme d'API, vous vous rendrez compte qu'il est rentable de passer d'un ensemble d'API ad hoc à une stratégie efficace. En intégrant ces pratiques à votre programme d'API, vous établirez un processus de conception et de production d'API prévisible, ainsi qu'une architecture complète qui garantit la gestion, la surveillance et la sécurisation de vos API.

 [Retour au début](#)

Ressources complémentaires

« [Modèles de conception cloud](#) » : des modèles de conception sont utiles pour créer des applications fiables, évolutives et sécurisées dans le cloud.

« [Le modèle de passerelle API et la communication directe entre clients et microservices](#) » : optimisation des interactions grâce à une architecture basée sur les microservices.

« [Une approche pratique de la conception d'API](#) » : guide pour tout savoir sur les API REST et en concevoir.

« [Conception d'API pour les microservices](#) » : introduction aux considérations relatives à la création d'API destinées aux microservices.